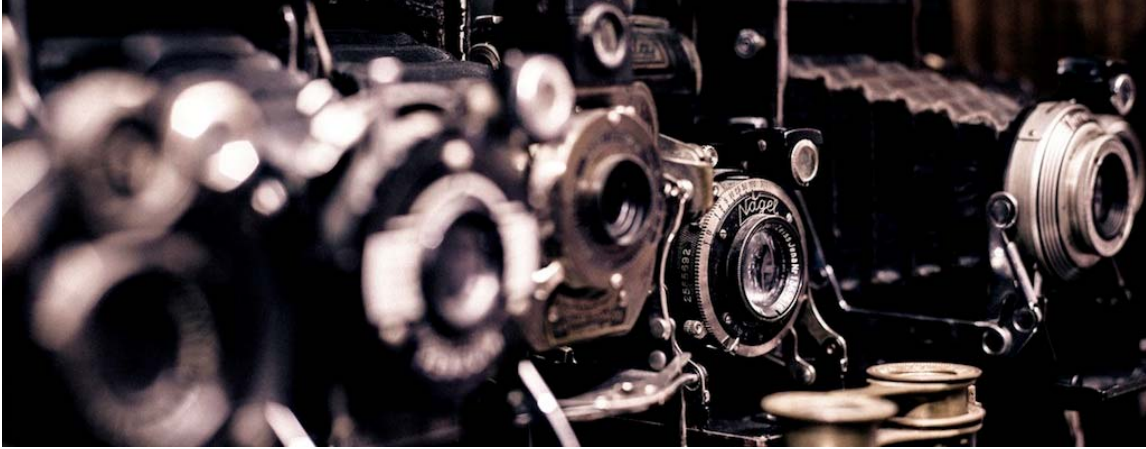


Geometric camera models

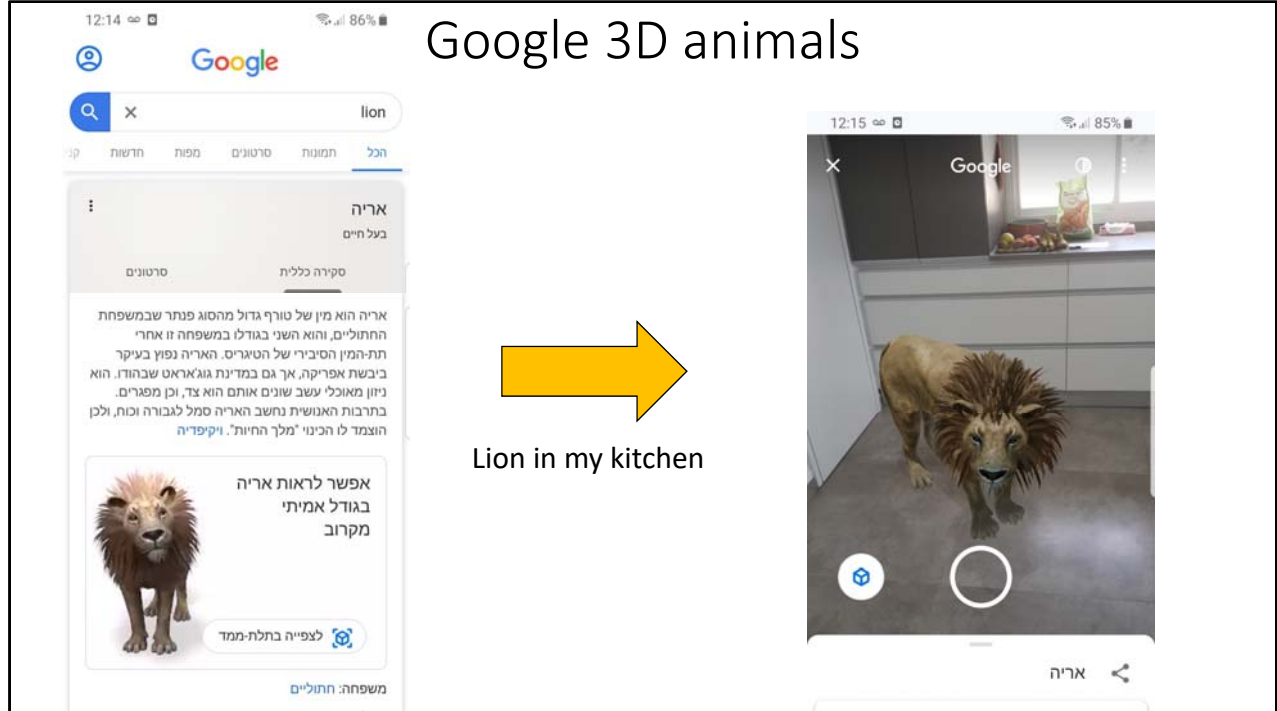


Slide credits: Ioannis Gkioulekas, Kris Kitani, Fredo Durand

Overview of today's lecture

- Some motivational imaging experiments.
- Pinhole camera.
- Accidental pinholes.
- Camera matrix.
- Perspective.
- Pose estimation.

Google 3D animals



12:14 86%

Google

lion

הכל תמונות סרטונים מפות חדשות קני

אריה

בעל חיים

סרטונים סקירה כללית

אריה הוא מין של טורף גדול מהסוג פנתר שבמשפחת החתוליים, והוא השני בגודלו במשפחה זו אחרי תת-המין הסיבירי של הטגריס. האריה נפוץ בעיקר ביבשת אפריקה, אך גם במדינת גואטמלה שבמרכז אמריקה. הוא מאוכלי עשב שונים אותם הוא צד, וכן מפגרים. בתרבות האנושית נחשב האריה סמל לגבורה וכות, ולכן הוצמד לו הכינוי "מלך החיות". ויקיפדיה



אפשר לראות אריה בגודל אמיתי מקרוב

לצפייה בתלת-ממד

משפחה: חתוליים

Google 3D animals

My nephews practicing their computer vision skills during covid19 pandemic



4

Google 3D animals



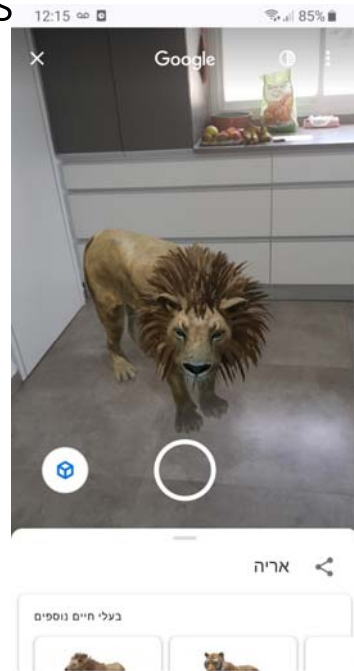
Where is the floor?

+



How to rotate?
How to scale?
Where in image
to place?

=



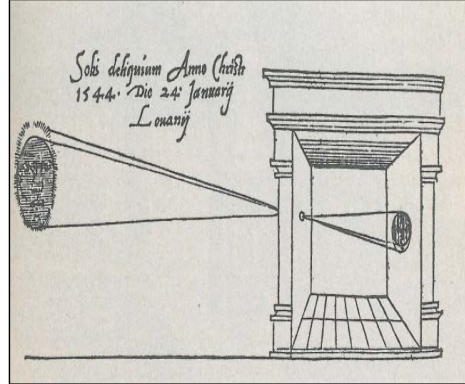
Pinhole camera a.k.a. camera obscura

First mention ...



Chinese philosopher Mozi
(470 to 390 BC)

First camera ...

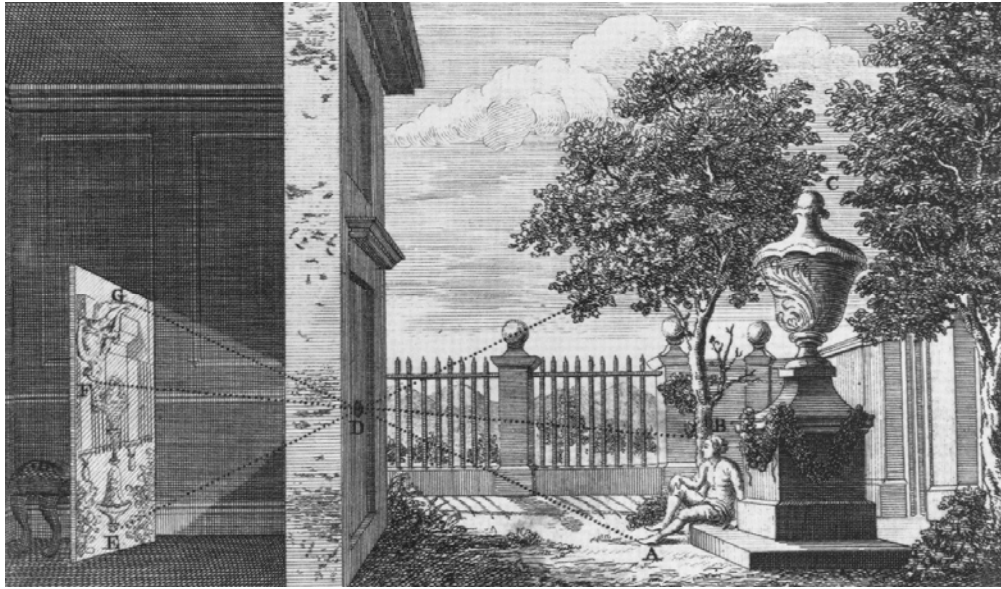


Greek philosopher Aristotle
(384 to 322 BC)

we will understand and describe mathematically how cameras produce two-dimensional images of the 3D world.

Our focus will be on a specific idealized camera model that we will call the pinhole camera.

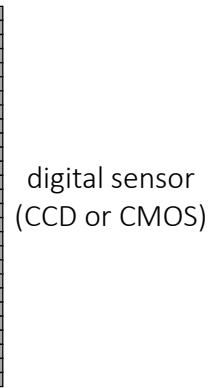
This type of camera, also known as camera obscura, has been known since ancient times, described by both Chinese and Greek philosophers.





A reason why pinhole cameras have been known for so long is that they tend to occur naturally, any time light is projected on a wall or other flat surface after passing through a small hole. These so-called *accidental pinholes* can produce compelling images. This example shows an image of the New Royal Palace at Prague Castle, formed on an attic wall.

Let's say we have a sensor...



Let's now look more concretely at how a pinhole camera model works.

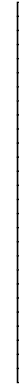
Let's assume that we are given a digital sensor, like the one you can find in any modern camera.

... and an object we like to photograph

real-world
object



digital sensor
(CCD or CMOS)



What would an image taken like this look like?

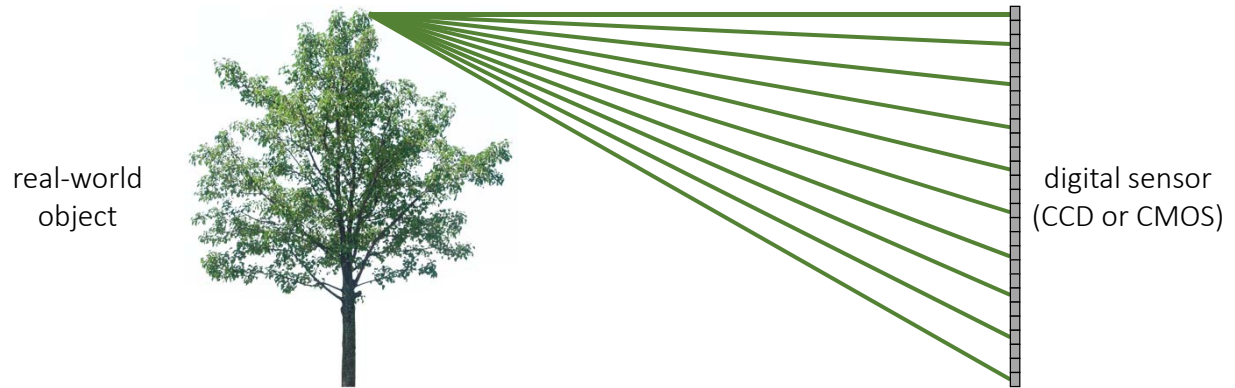
We want to use this sensor to capture a photograph of some real-world object.

Bare-sensor imaging

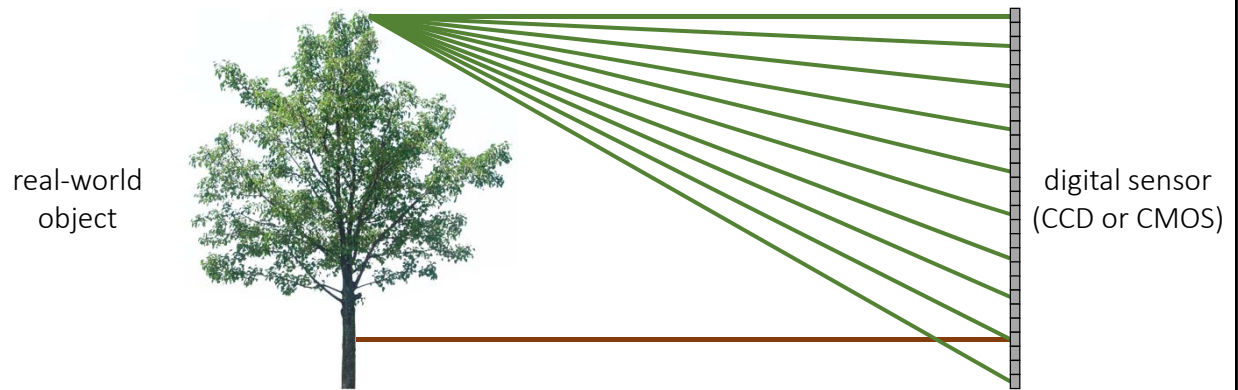


One way to do this is to simply expose the sensor without any optics. We call this bare-sensor imaging. In this case, all points on the object send light to all sensor pixels.

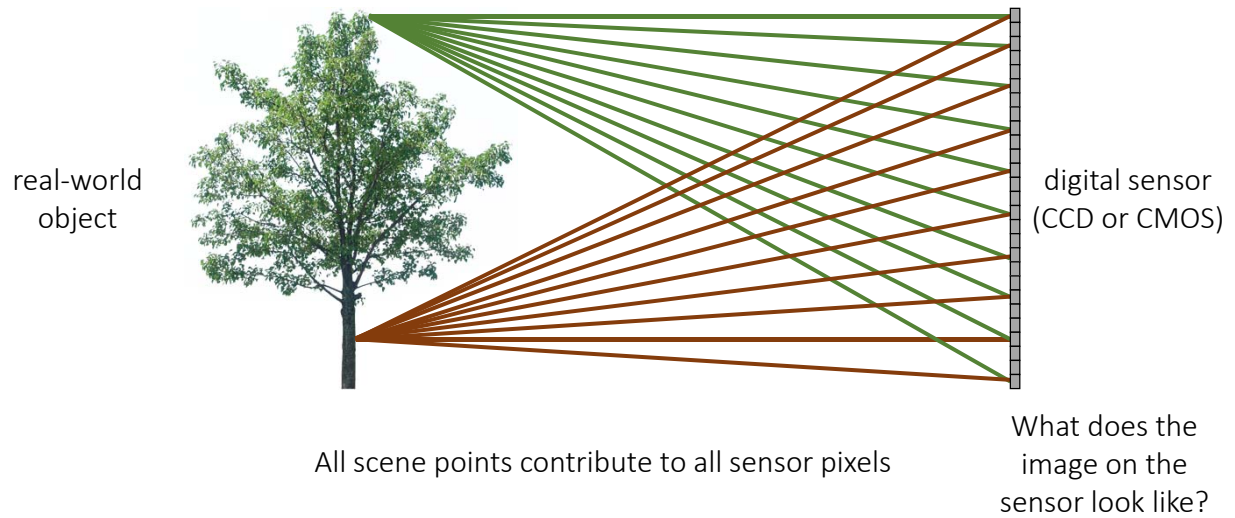
Bare-sensor imaging



Bare-sensor imaging



Bare-sensor imaging



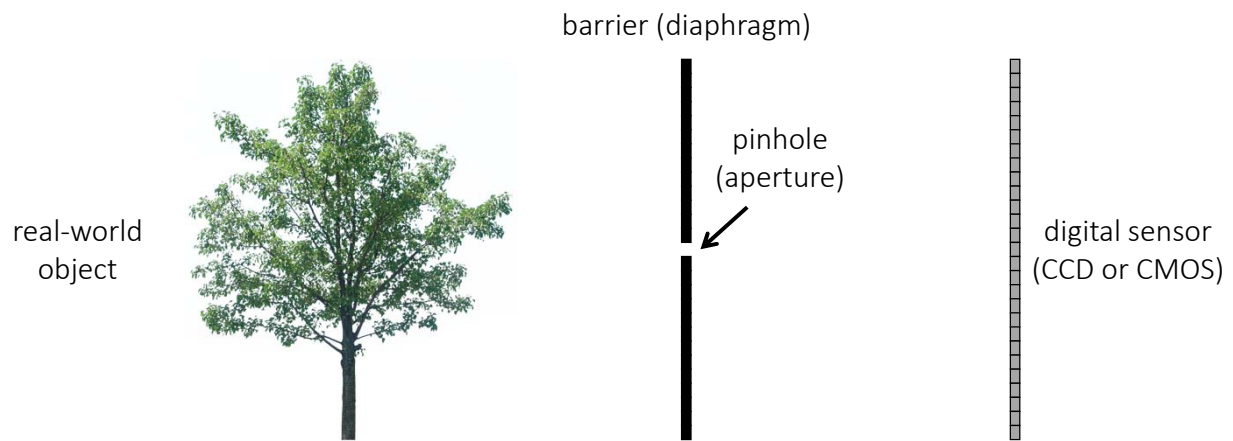
Bare-sensor imaging



All scene points contribute to all sensor pixels

The result will be that the sensor will capture a very blurry image such as this, where we cannot make out any of the details of the real-world object. This is obviously not what we want!

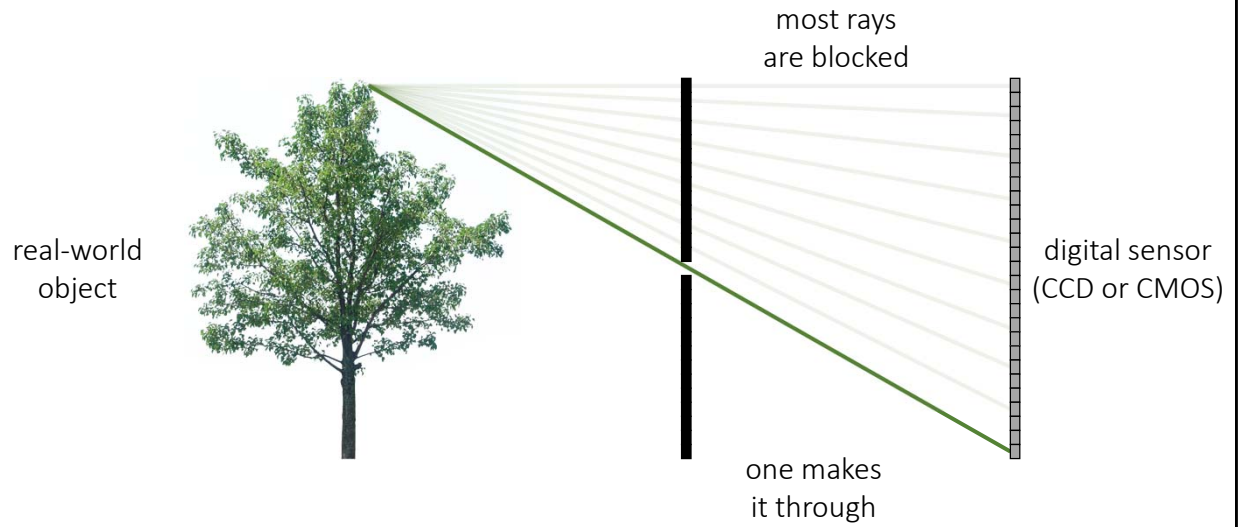
Let's add something to this scene

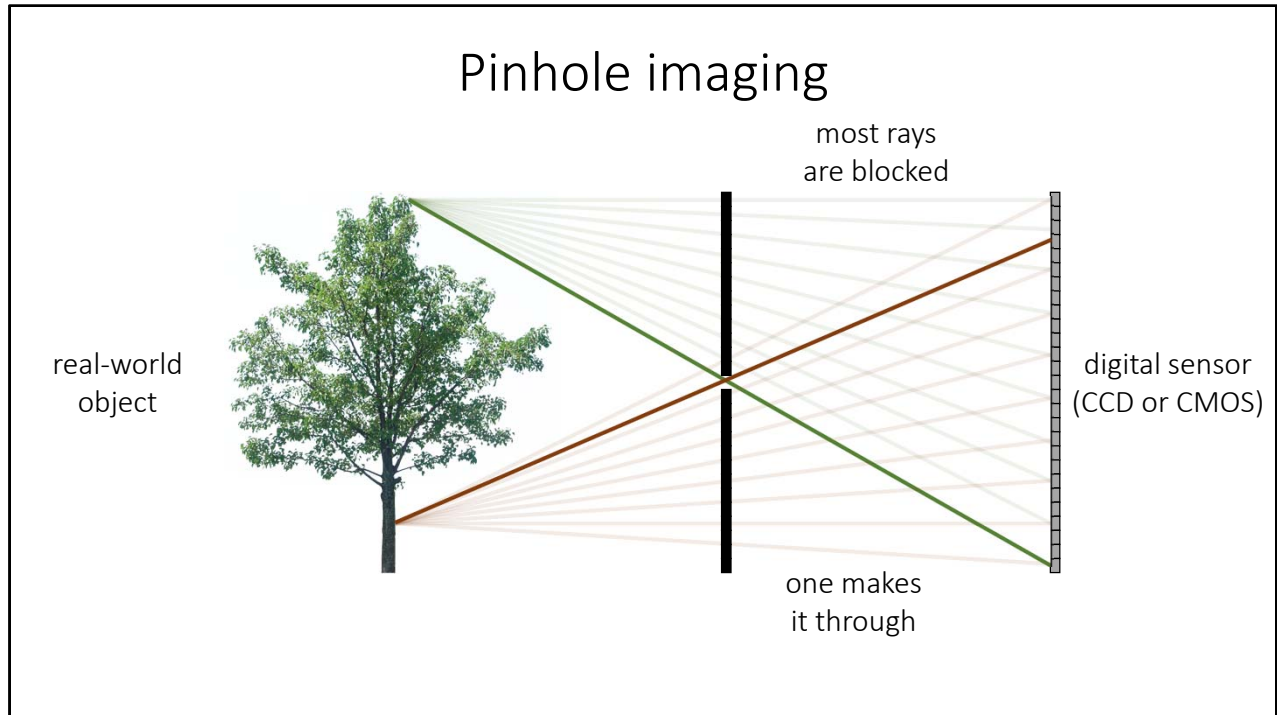


What would an image taken like this look like?

To fix this issue, we can go back to our scene and make the following change: We place a barrier between the sensor and the real-world object, and we pinch a small hole in the barrier.

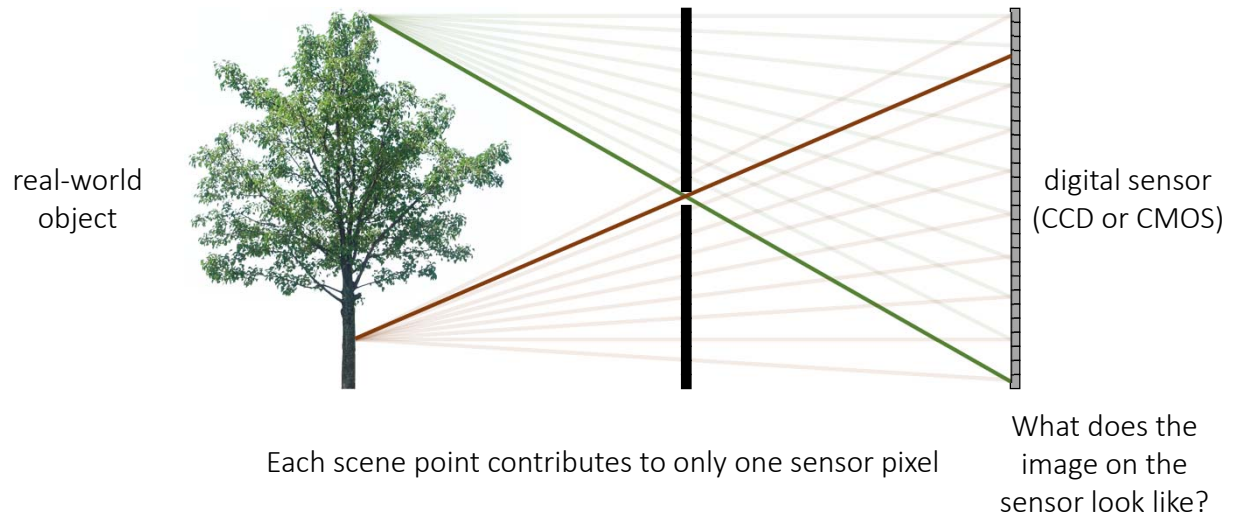
Pinhole imaging

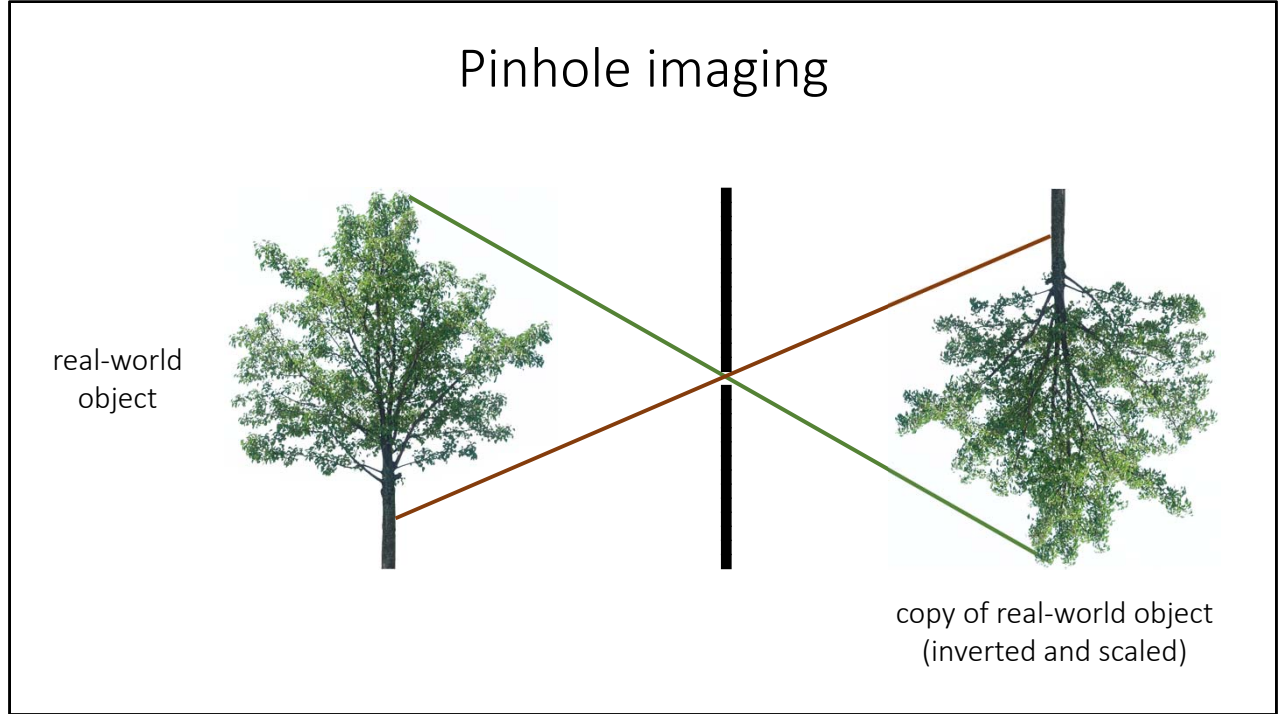




The barrier will block most rays from the object to the sensor, except for rays passing through the pinhole. The overall effect is that each point on the object sends light to only one pixel. We call this pinhole imaging.

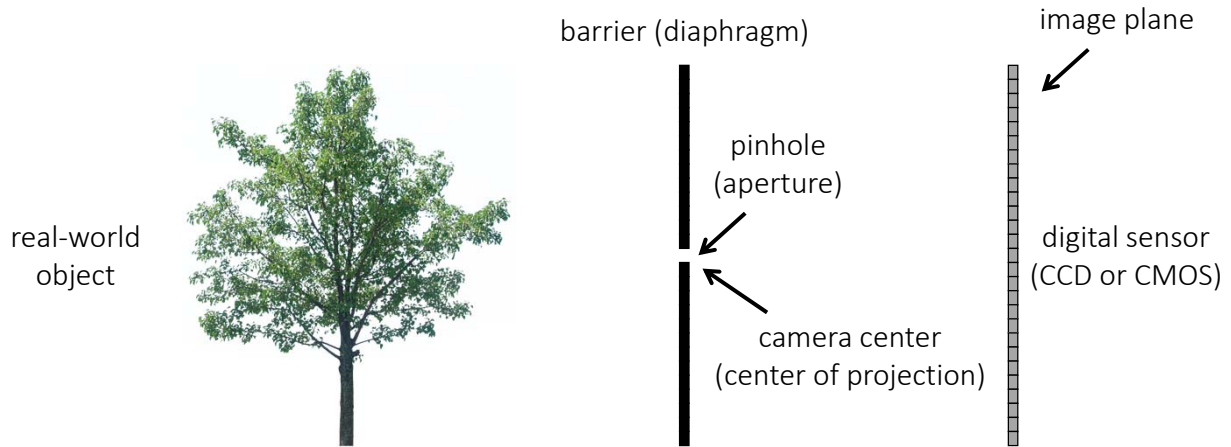
Pinhole imaging





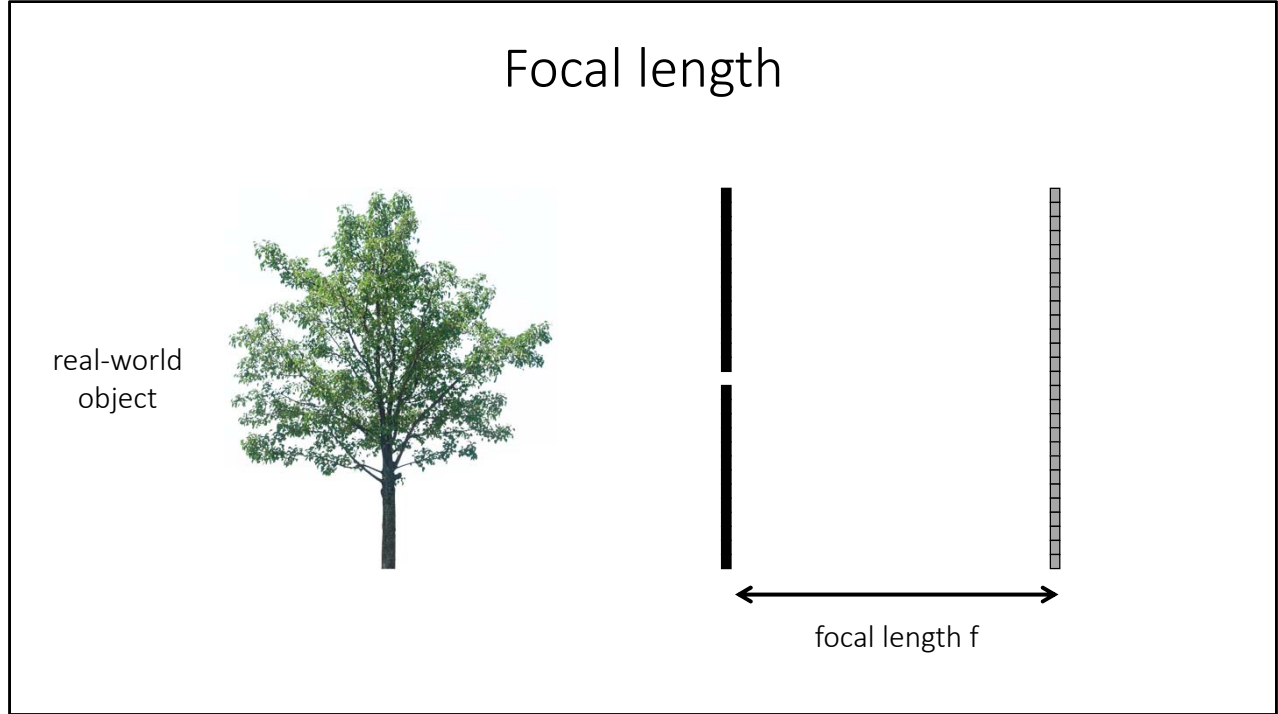
We now capture a clear image of the real-world object, albeit inverted and scaled.

Pinhole camera terms



Alright, let's look at this pinhole camera more closely and introduce some terminology. We have already identified the barrier and pinhole, which we also call diaphragm and aperture, respectively. We refer to the pinhole also as the camera center, for reasons we will explain later.

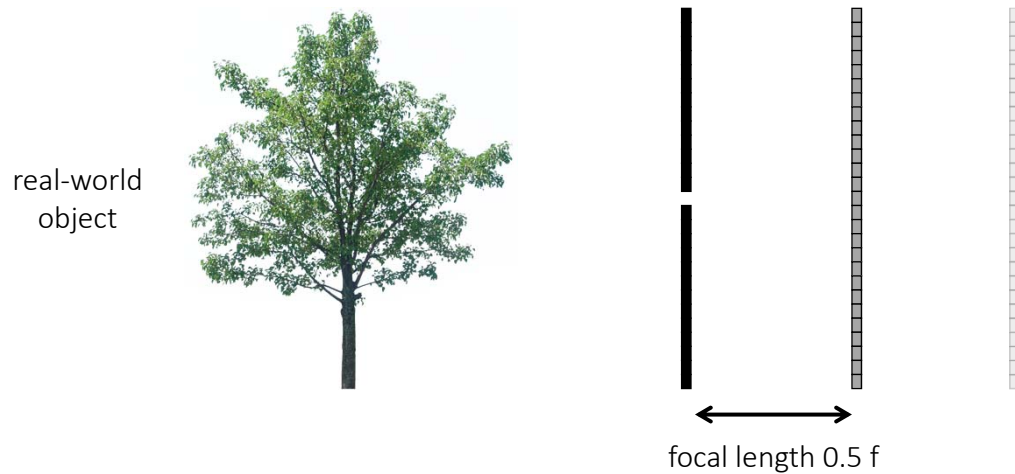
Lastly, we call the plane occupied by the digital sensor the image plane, as that is where the image is formed.



An important setting of the pinhole camera is its focal length, which is nothing else but the distance between the diaphragm and sensor plane.

Focal length

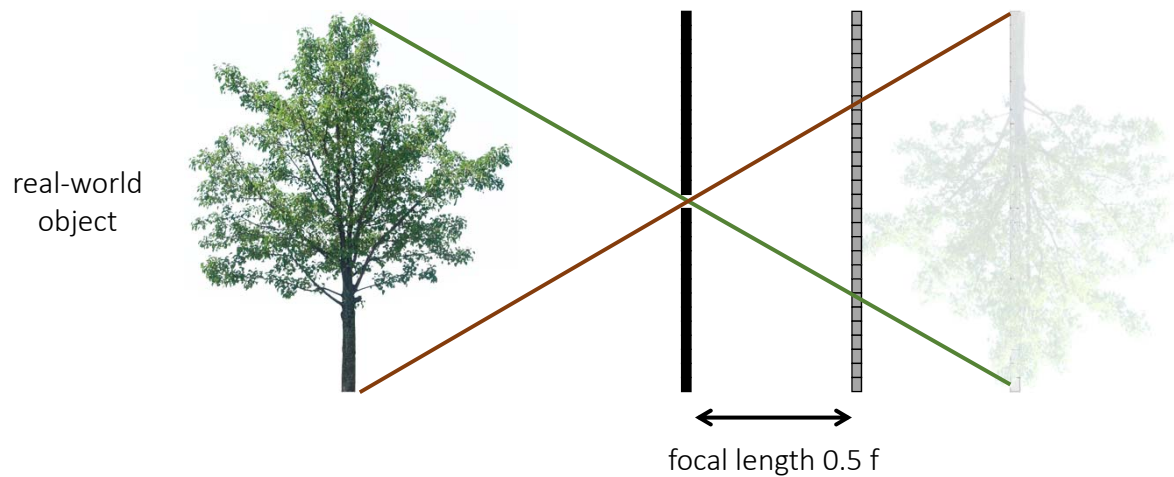
What happens as we change the focal length?



To understand how the focal length affects the images we capture, let's see what happens when we reduce it to half its value.

Focal length

What happens as we change the focal length?



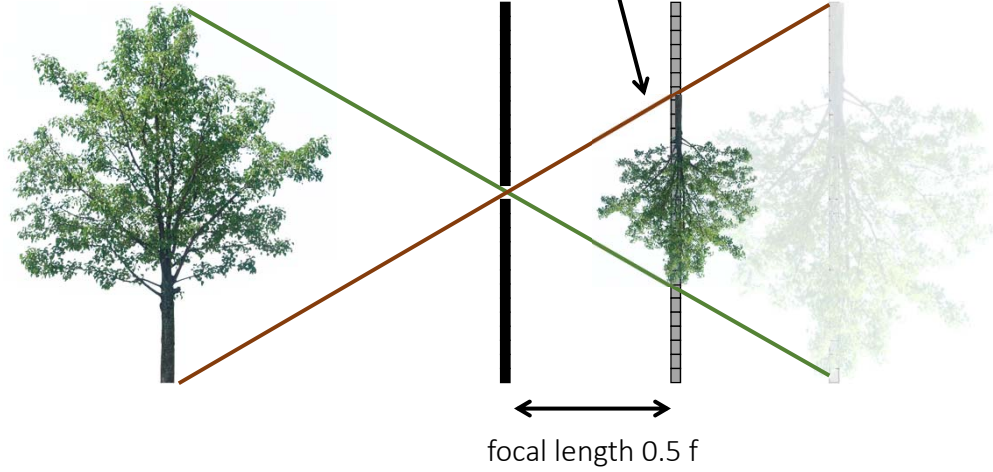
By tracing rays as before, we see that the result is that our image is scaled to half its original size.

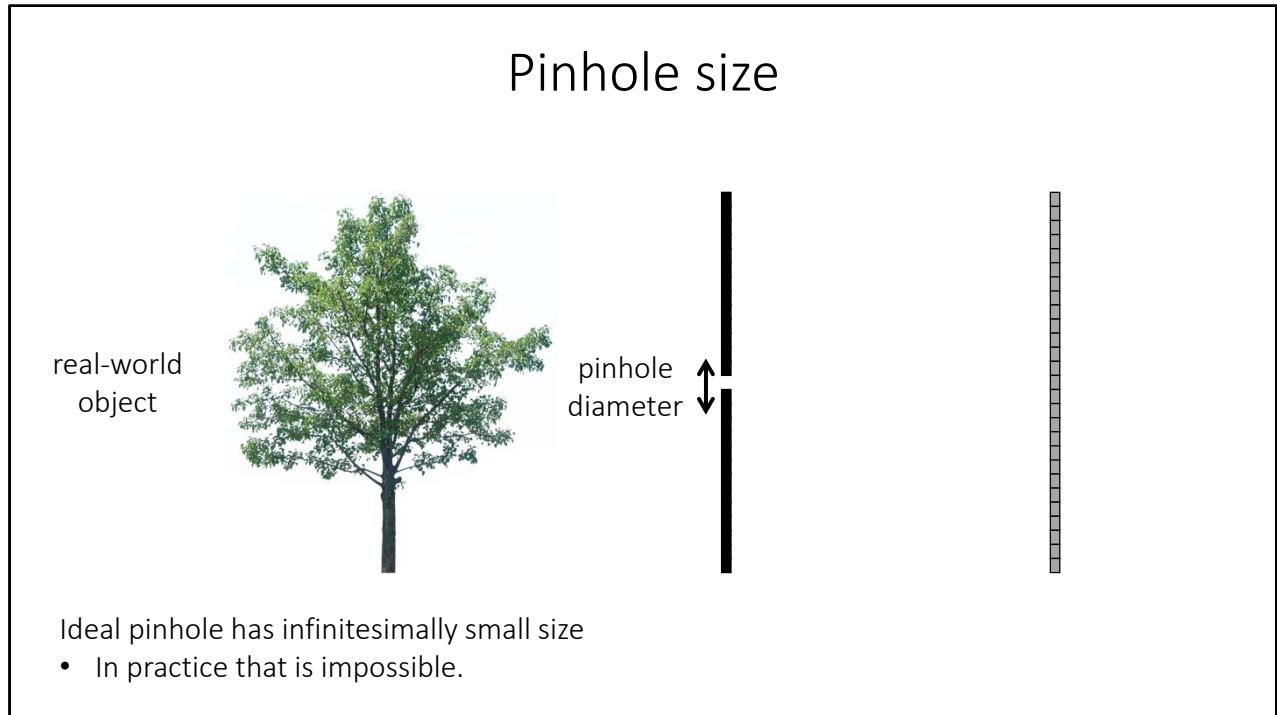
Focal length

What happens as we change the focal length?

object projection is half the size

real-world
object



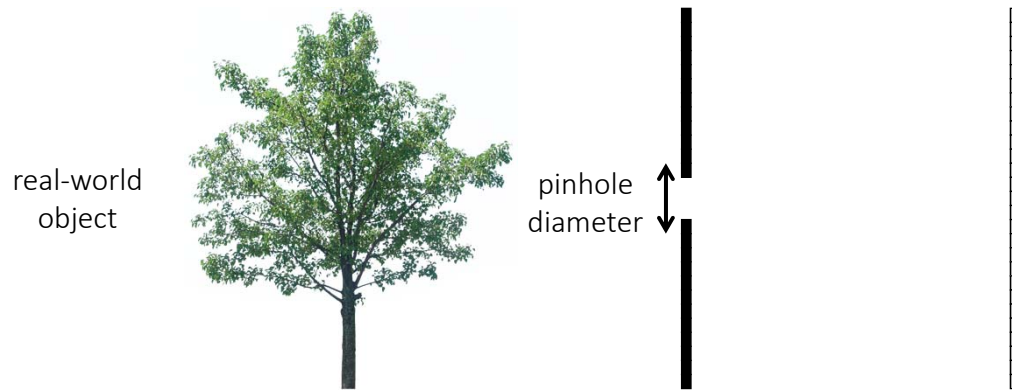


Another important setting of the pinhole camera is the diameter of the pinhole. In an ideal pinhole camera, the pinhole is infinitesimally small. However, this is not possible in practice, and the pinhole has some finite small size.

small size is a trade-off

Pinhole size

What happens as we change the pinhole diameter?

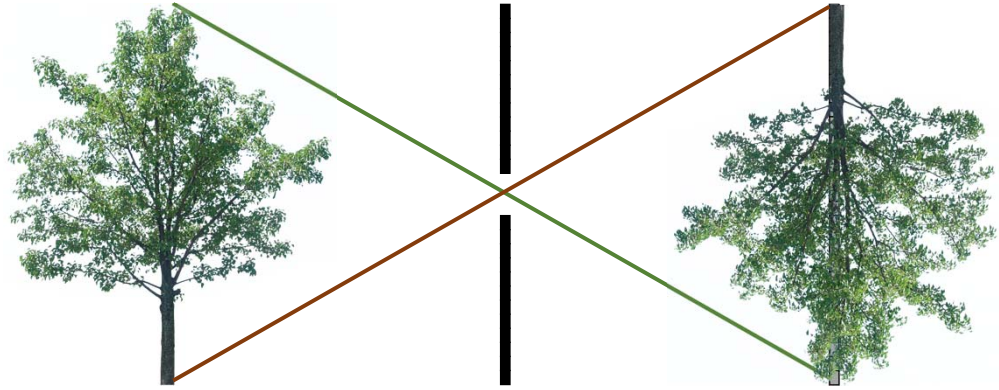


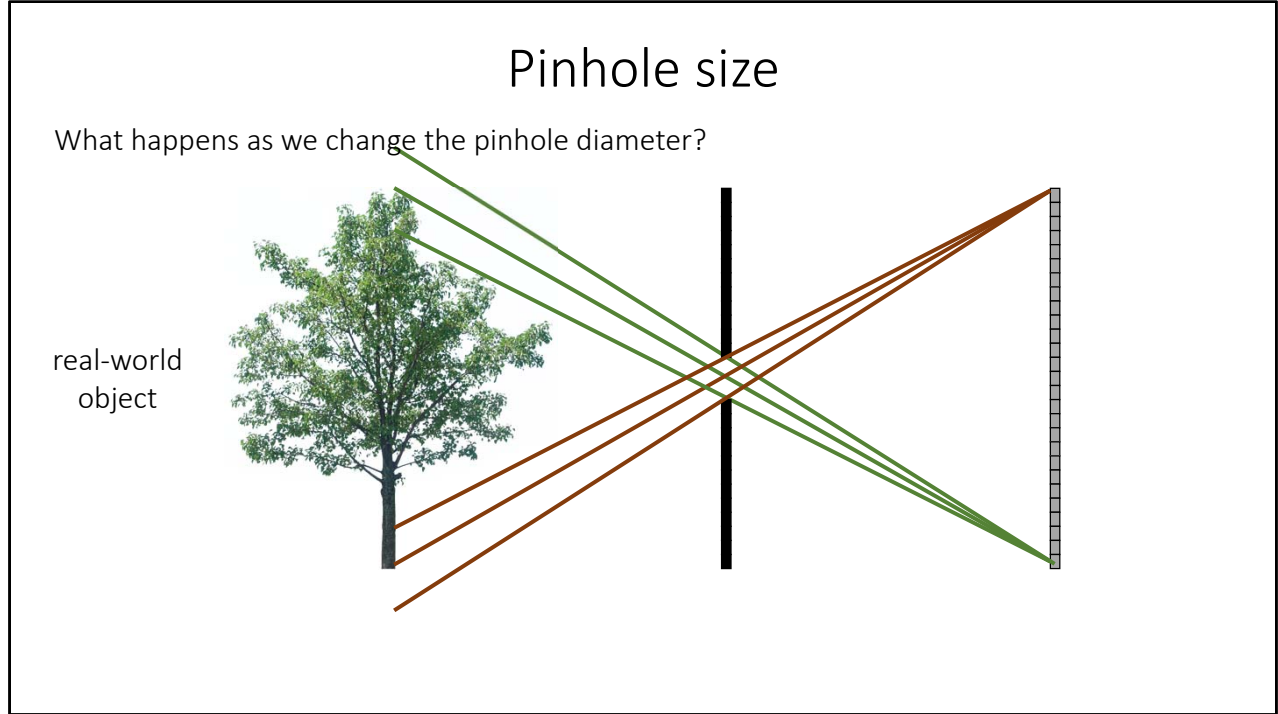
To understand the effect of the pinhole size, let's see what happens if we make the pinhole diameter larger.

Pinhole size

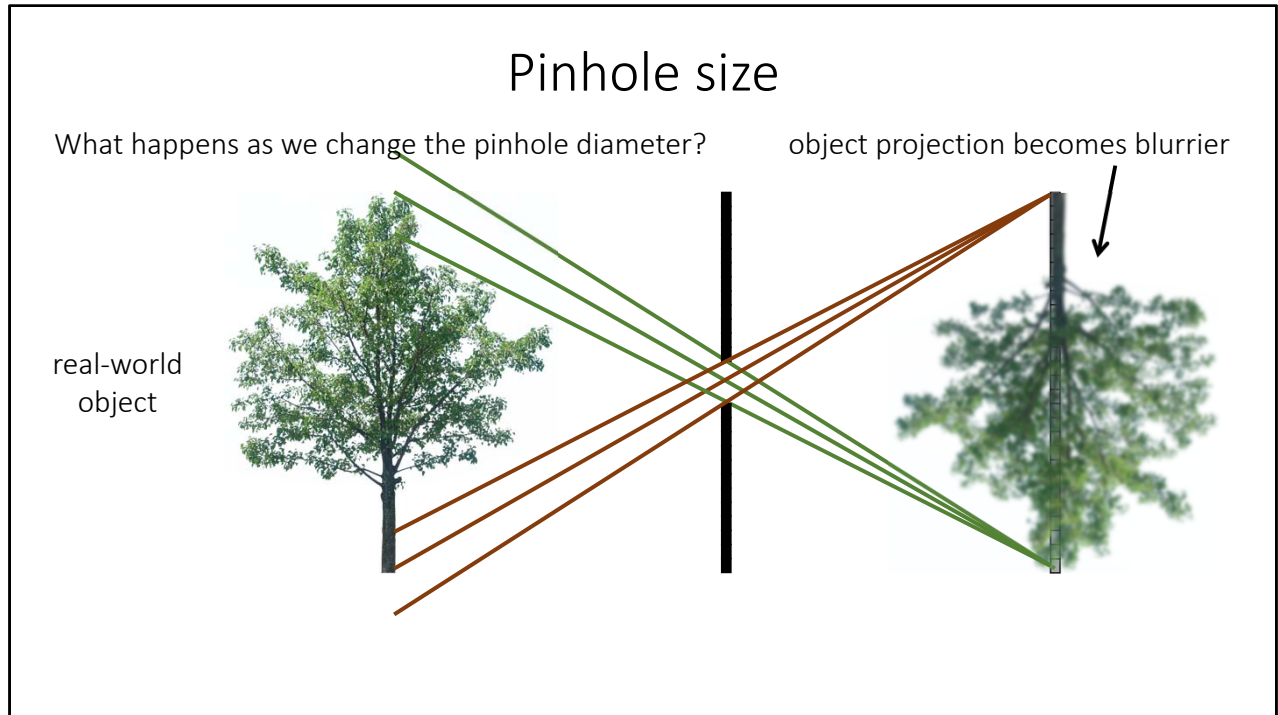
What happens as we change the pinhole diameter?

real-world
object



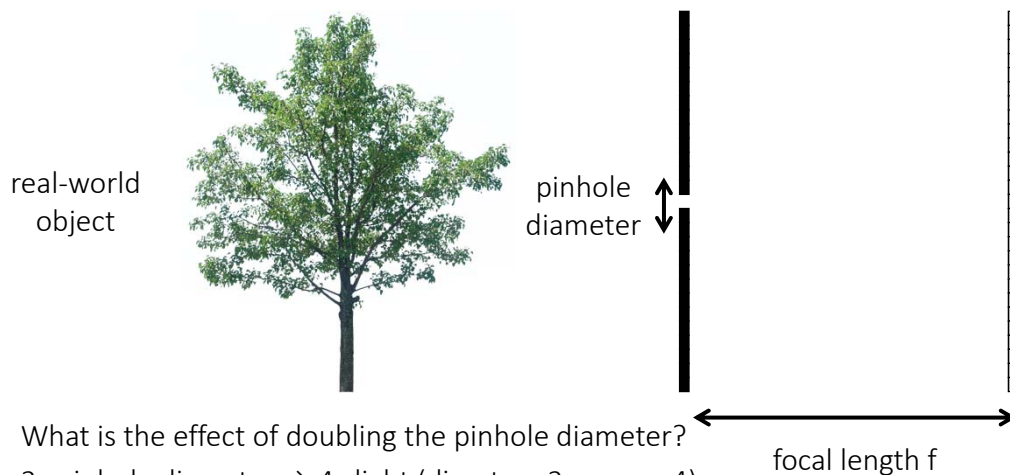


We see now that every pixel on the sensor receives rays from more than one point in the real-world object.



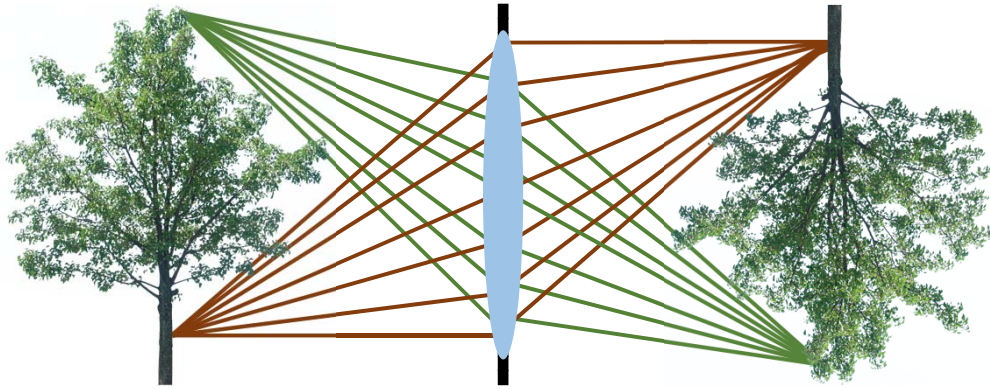
The result is that our image becomes blurrier, though not as blurry as in bare-sensor imaging.

What about light efficiency?



This suggests that, to make our images sharper, we need to make the pinhole as small as possible. However, we need to take into account that the smaller the pinhole, the more light we are blocking, resulting in images that are very dim and noisy. We see that pinhole cameras suffer from a trade-off between increasing image sharpness and improving light efficiency.

The lens camera



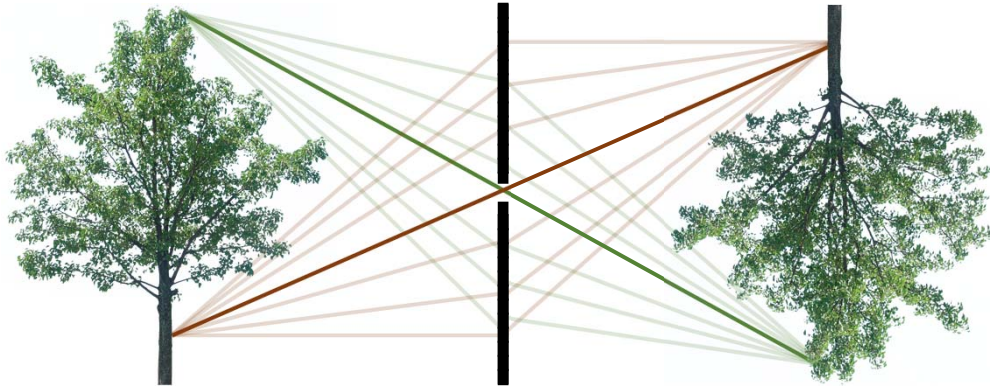
Lenses map “bundles” of rays from points on the scene to the sensor.

How does this mapping work exactly?

This is a good opportunity to briefly look at a different type of camera, which is the most common type of digital camera nowadays.

In particular, we can consider lens cameras, where we place a lens between the object and sensor. A lens is a piece of glass or other transparent material that uses refraction to bend light rays. Its shape is such that it maps bundles of rays from a point on the 3D object to a pixel on the sensor. This makes it possible to capture images that are both sharp and well-lit, which is the reason why we prefer to use lens-based cameras.

The pinhole camera

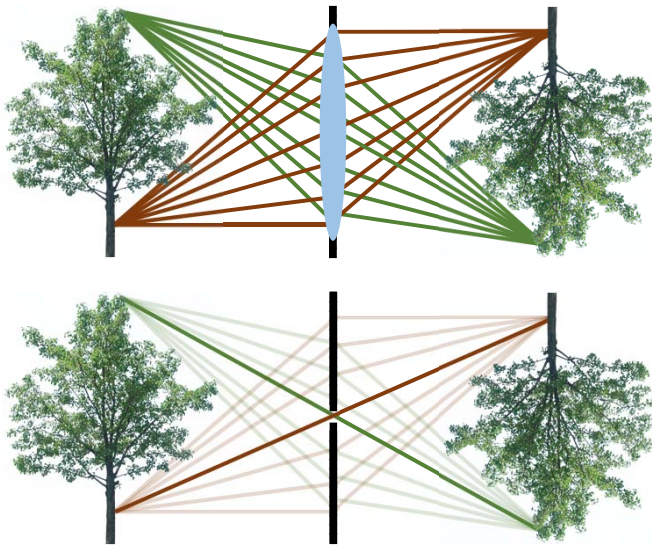


Central rays propagate in the same way for both models!

To see how a lens and pinhole camera compare to each other, let's look at the corresponding ray diagram for a pinhole camera. We note that rays passing through the center of the lens in the lens camera behave the same as rays passing through the pinhole in the pinhole camera.

This property makes it possible to use geometry to derive mathematical models for pinhole cameras that also hold for lens-based cameras. At the same time, using pinhole cameras simplifies our analysis, as it means that we do not need to worry about effects such as defocus.

Describing both lens and pinhole cameras



We can derive properties and descriptions that hold for both camera models if:

- We use only central rays.
- We assume the lens camera is in focus.

Mathematical camera description

We will now look at how to formalize and generalize our description of the pinhole camera.

2D Homogeneous coordinates reminder

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- homogeneous $\rightarrow$ heterogeneous

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow \begin{bmatrix} x/w \\ y/w \end{bmatrix}$$

- scale invariance

$$\begin{bmatrix} x & y & w \end{bmatrix}^T = \lambda \begin{bmatrix} x & y & w \end{bmatrix}^T$$

Special points:

- point at infinity

$$\begin{bmatrix} x & y & 0 \end{bmatrix}$$

- undefined

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

Before that just a quick reminder of homogeneous coordinates we are going to reuse.

3D Homogeneous coordinates

Conversion:

- heterogeneous $\rightarrow$ homogeneous

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \Rightarrow \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

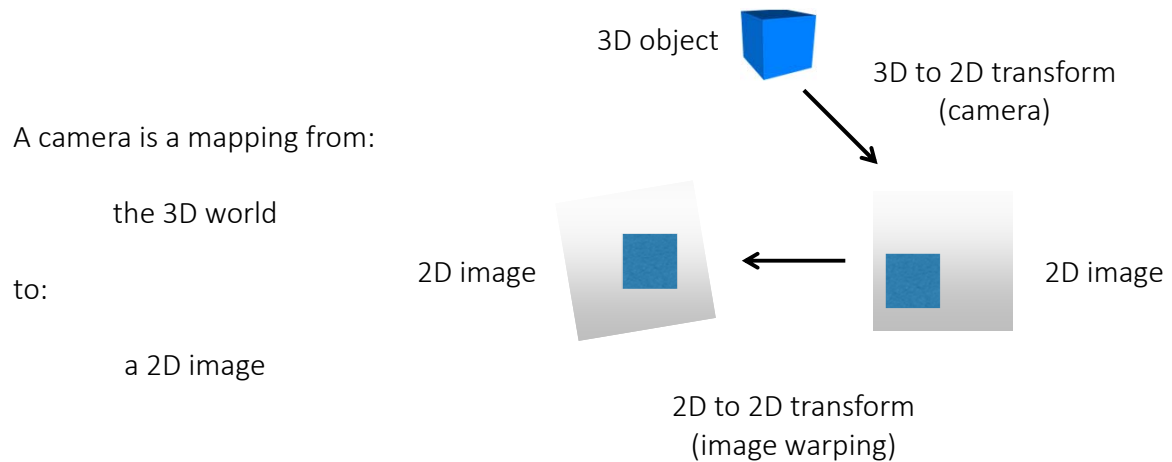
- homogeneous $\rightarrow$ heterogeneous

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} \Rightarrow \begin{bmatrix} X/W \\ Y/W \\ Z/W \end{bmatrix}$$

- scale invariance $\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \lambda \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix}$

Same definition in 3D

The camera as a coordinate transformation



We will now look at how to formalize and generalize our description of the pinhole camera.

In particular, we will treat the pinhole camera as a device that implements a mapping from points in the 3D world to points in a 2D image plane, potentially with an additional mapping to a different 2D image plane.

The camera as a coordinate transformation

A camera is a mapping from:

the 3D world

to:

a 2D image

$$\begin{array}{c} \text{homogeneous coordinates} \\ \swarrow \quad \searrow \\ \mathbf{p} = \mathbf{M} \mathbf{P} \\ \begin{array}{cc} \text{2D image point} & \begin{array}{cc} \text{camera} & \text{3D world} \\ \text{matrix} & \text{point} \end{array} \end{array} \end{array}$$

What are the dimensions of each variable?

In homogeneous coordinates, this viewpoint allows us to treat the pinhole camera as a matrix mapping 4x1 3D homogeneous coordinates to 3x1 2D homogeneous coordinates. In the rest of this lecture, we will derive the form of this 3x4 *camera matrix*.

The camera as a coordinate transformation

$$p = MP$$

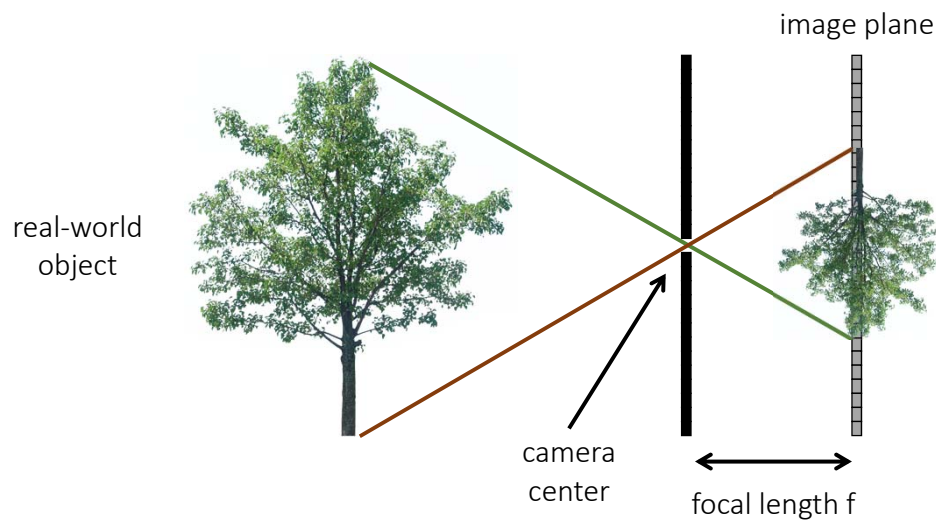
$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

homogeneous
image coordinates
3 x 1

camera
matrix
3 x 4

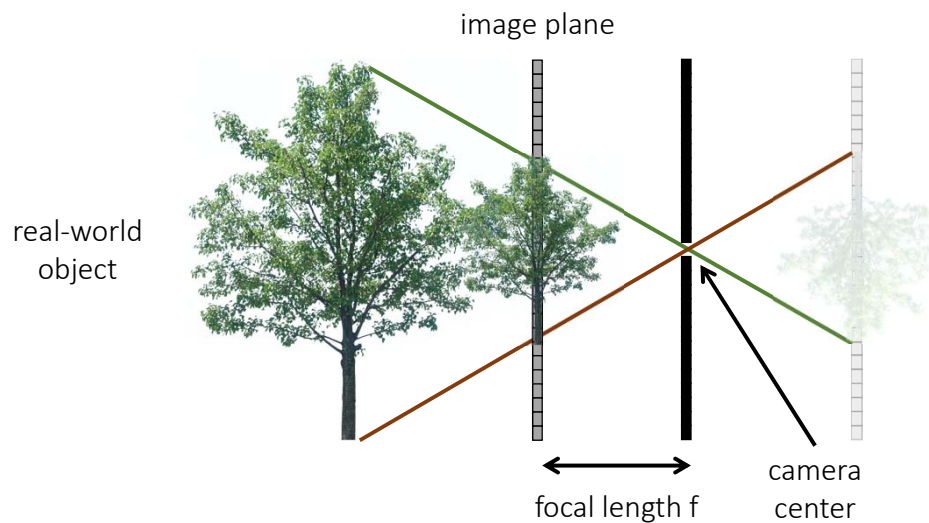
homogeneous
world coordinates
4 x 1

The pinhole camera



To derive the camera matrix, let's go back to our simple pinhole camera model.

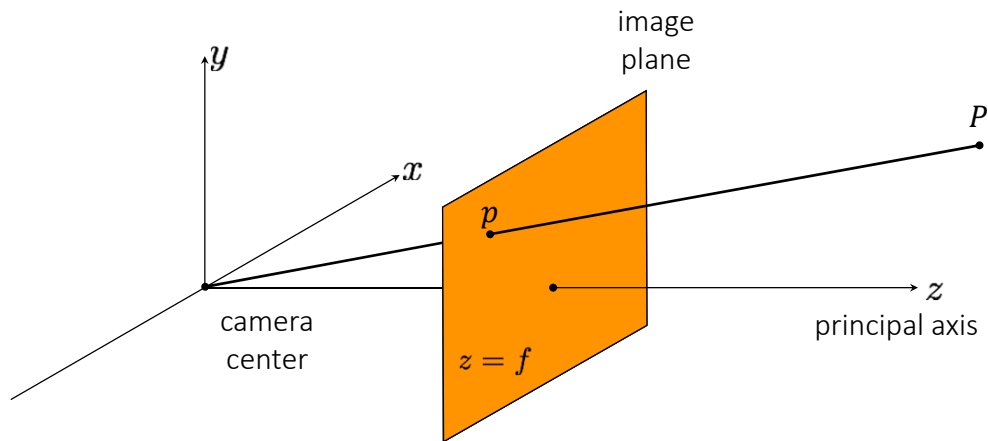
The (rearranged) pinhole camera



To simplify our derivation, we will make a change in our model of the pinhole camera. We will assume that the image plane is placed between the 3D scene and the diaphragm, rather than after the diaphragm. This is *not* how the pinhole camera is built in reality. But using this rearranged model simplifies our analysis, as we no longer need to take into account the inversion in the image formation.

(עניין) → נניח כי ה'פ' הוא בין ה'א' ל'ב'

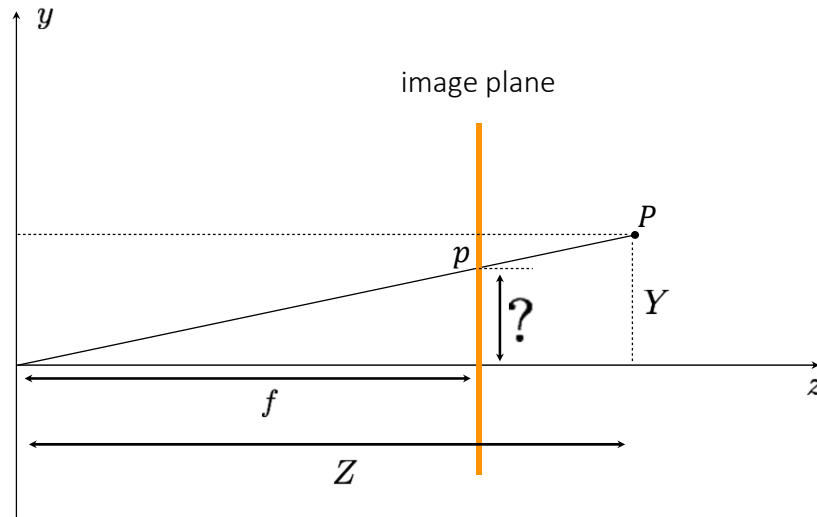
The (rearranged) pinhole camera



What is the equation for image coordinate $\mathbf{p}$ in terms of $\mathbf{P}$?

This is what this rearranged model looks like in 3D. We see that every 3D point upper-case P is mapped to a 2D point lower-case p , by first forming the line that connects the 3D point to the camera center, then finding where this line intersects the image plane. Let's see what the equation for this mapping is exactly.

The 2D view of the (rearranged) pinhole camera

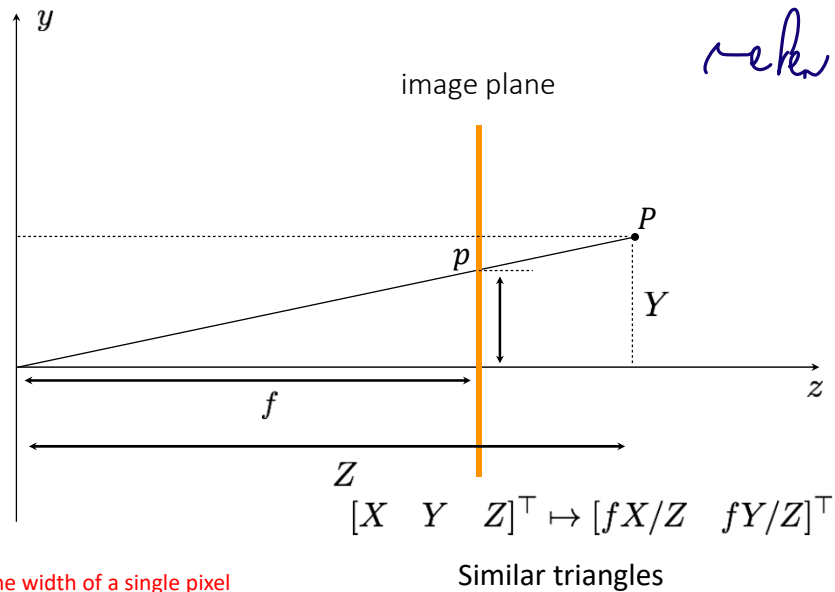


What is the equation for image coordinate p in terms of P ?

Units note: "1" is the width of a single pixel

To derive the exact equation, it is easier to draw a 2D view of the re-arranged pinhole camera, as shown here.

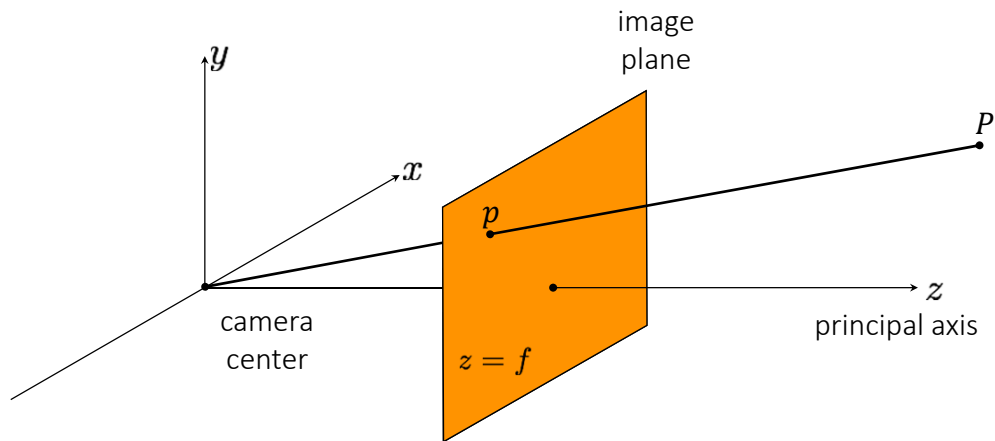
The 2D view of the (rearranged) pinhole camera



Units note: "1" is the width of a single pixel

Then, using similar triangles, it is easy to see that the coordinates of the 2D image point equal the X,Y coordinates of the 3D point, divided by its Z coordinate and scaled by the focal length which is the distance of the image plane from the origin.

The (rearranged) pinhole camera



What is the camera matrix $\mathbf{M}$ for a pinhole camera?

Units note: "1" is the width of a single pixel

$$p = MP$$

The pinhole camera matrix

Relationship from similar triangles:

$$[X \ Y \ Z]^\top \mapsto [fX/Z \ fY/Z]^\top$$

General camera model:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

What does the pinhole camera projection look like?

$$\mathbf{M} = \begin{bmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{bmatrix}$$

Units note: "1" is the width of a single pixel

This relationship is non-linear, because it uses the heterogeneous coordinates of the 3D and 2D points.

We can write the same projection relationship equivalently using homogeneous coordinates.

The pinhole camera matrix

Relationship from similar triangles:

$$\begin{bmatrix} X & Y & Z \end{bmatrix}^\top \mapsto \begin{bmatrix} fX/Z & fY/Z \end{bmatrix}^\top$$

General camera model:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

What does the pinhole camera projection look like?

$$\mathbf{M} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Units note: "1" is the width of a single pixel

Let's verify what this matrix is doing

$$\mathbf{M} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \begin{bmatrix} fX/Z \\ fY/Z \\ 1 \end{bmatrix}$$

Doing so allows us to express the projection relationship simply as a matrix-vector product with a special 3x4 matrix, that we call the *perspective projection matrix*.

Perspective distortion

Perspective distortion

The pinhole camera have “*perspective distortion*”.

$$M = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$[X \quad Y \quad Z]^T \mapsto [fX/Z \quad fY/Z]^T$$

Perspective projection from
(homogeneous) 3D to 2D coordinates

Perspective distortion: magnification
changes with depth

It is worth taking a closer look at the perspective projection matrix, to develop some intuition about the mapping it implements, and especially about a property of this mapping called perspective distortion.

The third column of the perspective projection matrix has the effect that, in heterogeneous coordinates, the location of the 2D point on the image plane is scaled by the depth Z of the 3D point. In other words, the magnification of an object decreases, as its distance from the pinhole camera increases.

Forced perspective



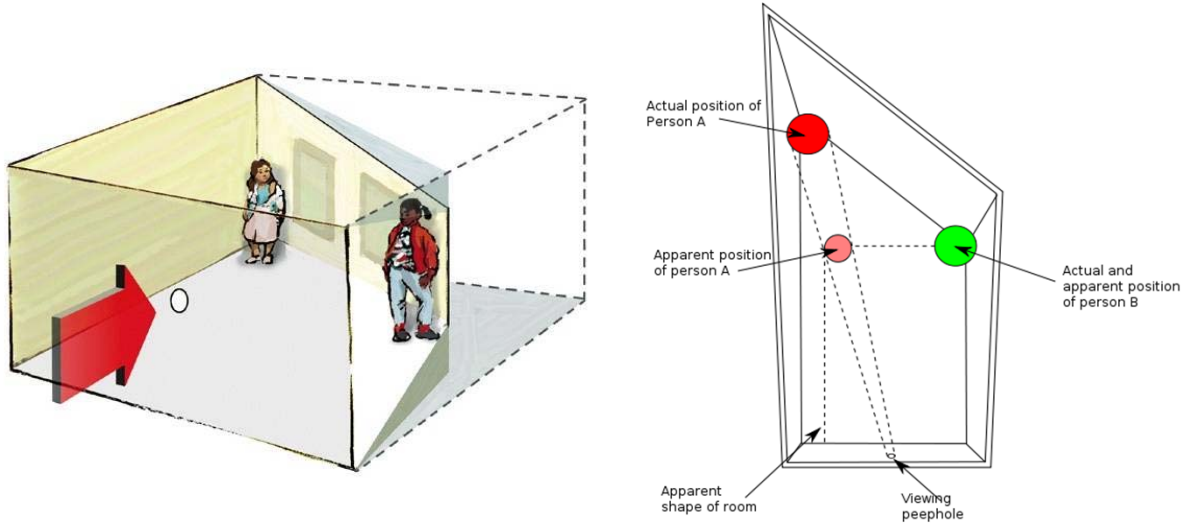
This property is fundamental in perspective projection, and can be used to produce all sorts of illusions and trick photographs. A common trick is to use the so-called forced perspective, to make two objects appear to be the same size, as in this photograph. What is really going on here is that the person in the photograph is a lot closer to the camera than the Eiffel tower. Because of perspective projection, the two are scaled differently on the image, and appear to have the same size.

The Ames room illusion



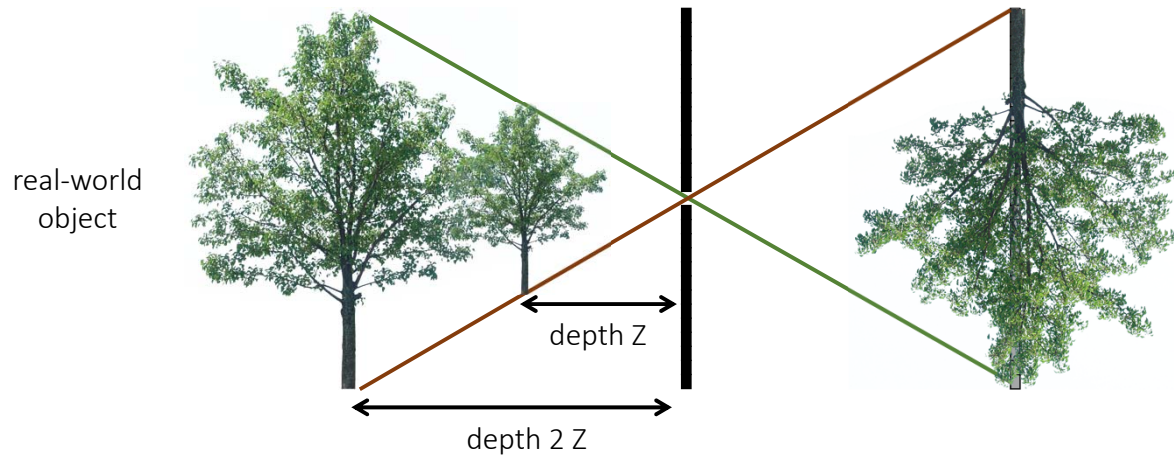
Another example is the so-called Ames room illusion.

The Ames room illusion

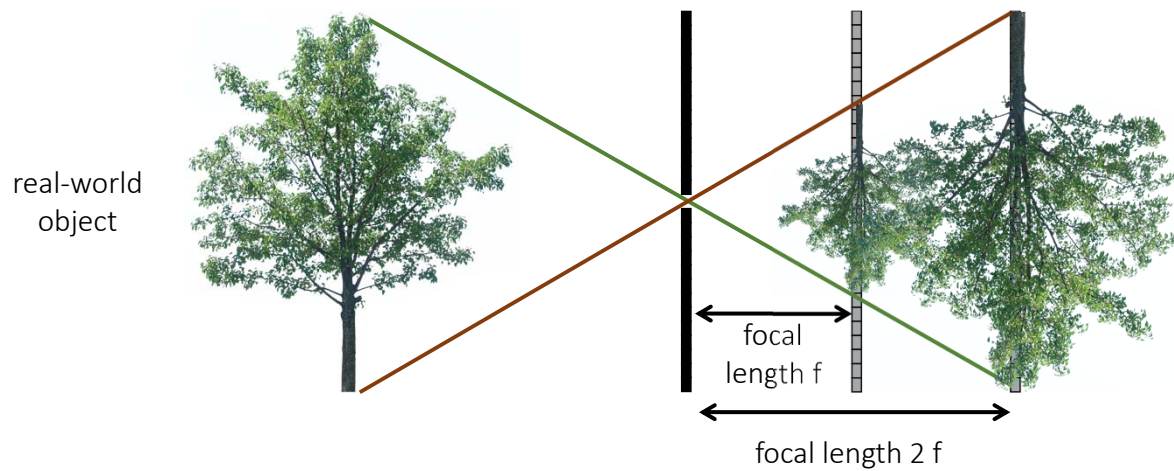


Magnification depends on depth

What happens as we change the focal length?

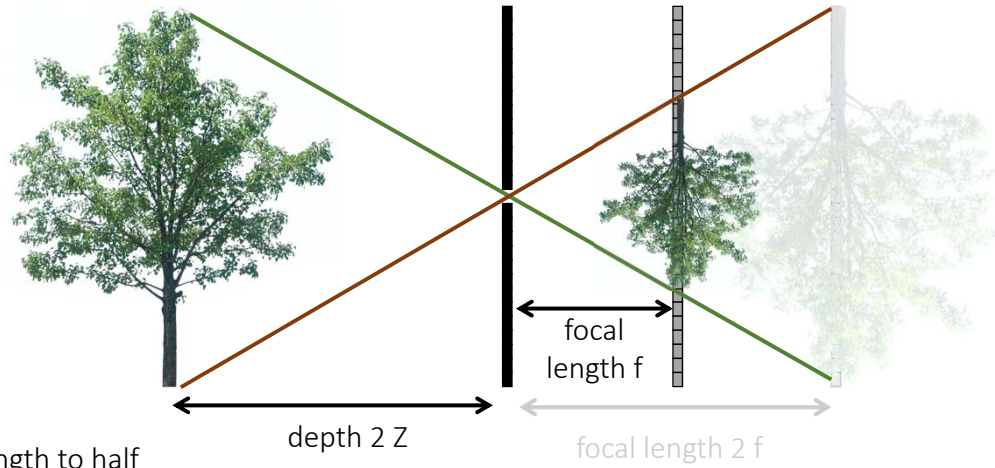


Magnification depends on focal length



What if...

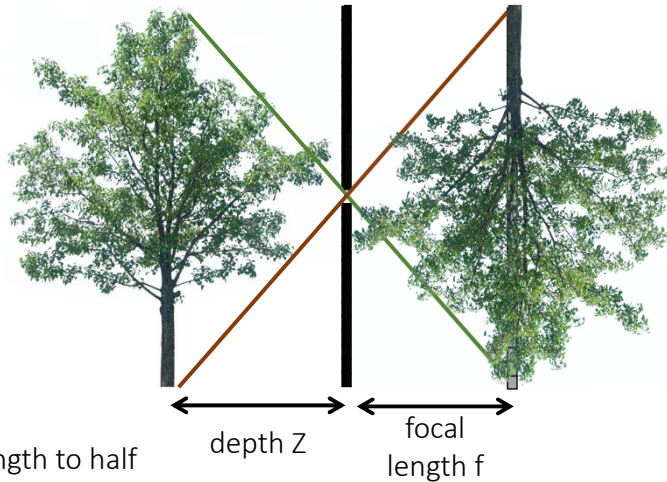
real-world
object



1. Set focal length to half

What if...

real-world
object



Is this the same image as
the one I had at focal
length $2f$ and distance $2Z$?

1. Set focal length to half
2. Set depth to half

התמונה מראה את האפקט של הפרספקטיבה על התמונה

Perspective distortion



long focal length



mid focal length



short focal length

Perspective distortion



Vertigo effect

Named after Alfred Hitchcock's movie

- also known as "dolly zoom"



Vertigo effect



How would you
create this effect?

Vertigo effect

$$\begin{bmatrix} X & Y & Z \end{bmatrix}^T \mapsto \begin{bmatrix} fX/Z & fY/Z \end{bmatrix}^T$$

Move camera center
back by αZ_o , set focal
length to αf

$$\begin{bmatrix} X & Y & Z \end{bmatrix}^T \rightarrow \begin{bmatrix} \frac{\alpha f X}{\alpha Z_o + (Z - Z_o)} & \frac{\alpha f Y}{\alpha Z_o + (Z - Z_o)} \end{bmatrix}^T$$

Fixed magnification for points at depth
 Z_o magnification changing for other depths

Vertigo effect

- A dolly zoom is an in-camera effect that appears to undermine normal visual perception.
- The effect is achieved by zooming a zoom lens to adjust the angle of view (often referred to as field of view, or FOV) while the camera dollies (moves) toward or away from the subject in such a way as to keep the subject the same size in the frame throughout. In its classic form, the camera angle is pulled away from a subject while the lens zooms in, or vice versa. Thus, during the zoom, there is a continuous perspective distortion, the most directly noticeable feature being that the background appears to change size relative to the subject.
- The visual appearance for the viewer is that either the background suddenly grows in size and detail and overwhelms the foreground, or the foreground becomes immense and dominates its previous setting, depending on which way the dolly zoom is executed. As the human visual system uses both size and perspective cues to judge the relative sizes of objects, seeing a perspective change without a size change is a highly unsettling effect, often with strong emotional impact.
- The effect was first conceived by Irmin Roberts, a Paramount second-unit cameraman, in Alfred Hitchcock's film Vertigo.^[1] The shot has since been used in many other films, including Goodfellas,^[2] Jaws,^[3] and the Lord of the Rings films.

The pinhole camera matrix

Relationship from similar triangles:

$$\begin{bmatrix} X & Y & Z \end{bmatrix}^\top \mapsto \begin{bmatrix} fX/Z & fY/Z \end{bmatrix}^\top$$

General camera model:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

What does the pinhole camera projection look like?

$$\mathbf{M} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Moving back to the mathematical modeling of camera matrices. We expressed projection relationship simply as a matrix-vector product with a special 3x4 matrix, that we call the *perspective projection matrix*.

Camera matrix decomposition

We can decompose the camera matrix like this:

$$M = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

(homogeneous) transformation
from 2D to 2D, accounting for not
unit focal length and origin shift

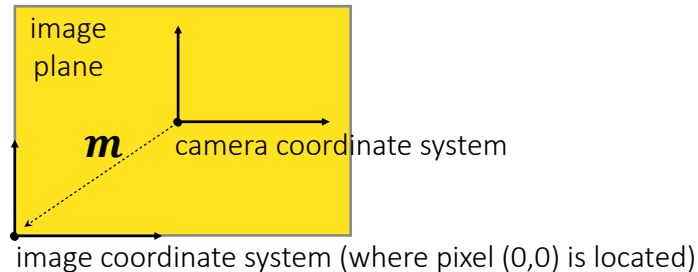
(homogeneous) projection from 3D
to 2D, assuming image plane at $f = 1$
and shared camera/image origin

Units note: "1" is the width of a single pixel

We can equivalently rewrite the new camera matrix as the product of a perspective projection matrix assuming unit focal length, with a matrix corresponding to a 2D scaling transformation, like those we covered in the lecture on 2D Transformations and Homographies. Effectively, the camera matrix first maps 3D points to 2D points, exactly the same as in the unit focal length case before, then scales the 2D points by an amount equal to the focal length.

Generalizing the camera matrix

In particular, the camera origin and image origin may be different:



How does the camera matrix change?

$$M = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

shift vector
transforming
camera origin to
image origin

Units note: "1" is the width of a single pixel

We will consider how to change the camera matrix, to account for changes in the properties of the pinhole camera.

One internal property of the camera we want to take into account when writing down the camera matrix is the coordinate system it uses for its image plane. Up until now, we have been assuming that the origin of the image plane is at the center of image.

However, many cameras parameterize pixels by using an origin corresponding to the corner in the image. We want to make sure that our camera matrix takes this into account.

In particular, we need to change the 2D transformation component of the camera matrix to include a translation. As we saw in the lecture on Transformations and Homographies, we can do this by adding the translation vector mapping the camera origin to the image origin at the third column of the 2D transformation matrix.

Camera matrix decomposition

We can decompose the camera matrix like this:

$$M = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & | & 0 \\ 0 & 1 & 0 & | & 0 \\ 0 & 0 & 1 & | & 0 \end{bmatrix}$$

(homogeneous) transformation
from 2D to 2D, accounting for not
unit focal length and origin shift

(homogeneous) projection from 3D
to 2D, assuming image plane at $f = 1$
and shared camera/image origin

Also written as: $M = K[I|0]$ where $K = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix}$ Intrinsic matrix.
Stay fixed when we move the camera

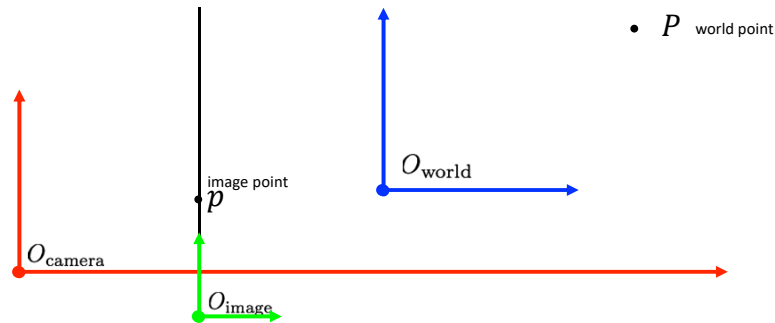
To summarize, when we take into account internal characteristics of the pinhole camera, we can decompose its 3x4 camera matrix into the product of two matrices.

The rightmost of these two matrices is the homogeneous perspective projection from 3D to 2D, assuming the basic pinhole camera of unit focal length. The leftmost of the two matrices is a homogeneous 2D transformation, shifting and scaling points on the image. The leftmost matrix depends only on internal properties of the camera, such as its focal length and origin. We will refer to these as the camera intrinsics.

The reason this separation is important because later on we will want to analyze how the camera matrix changes when the camera is moved. We will use the fact that the intrinsic parameters stay fixed, and only the rightmost matrix is changing.

Generalizing the camera matrix

In general, there are *three*, generally different, coordinate systems.

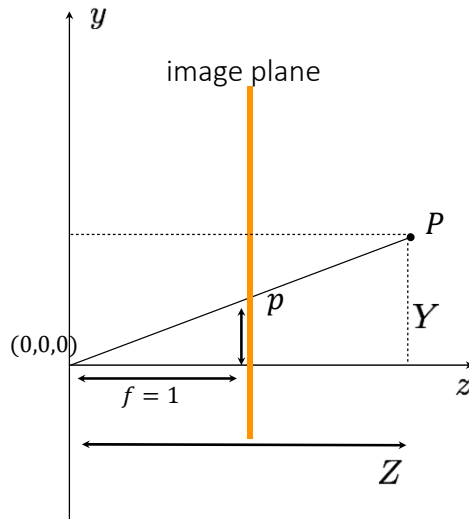


We need to know the transformations between them.

So far, we have seen how to modify the camera matrix to account for internal characteristics of the pinhole camera. We will now see how to modify the camera matrix to model how the camera relates to the world around it. To see what we mean by that, let's consider again the 3D drawing of the rearranged pinhole camera. The pinhole camera creates a coordinate system that we can use to describe any 3D point in the world. This coordinate system has its origin at the camera center, the pinhole. It has x and y axes running parallel to the image plane. And it has a z axis orthogonal to the image plane. The camera matrices we derived so far assume that the 3D points we project on the image plane are expressed in this camera-based 3D coordinate system.

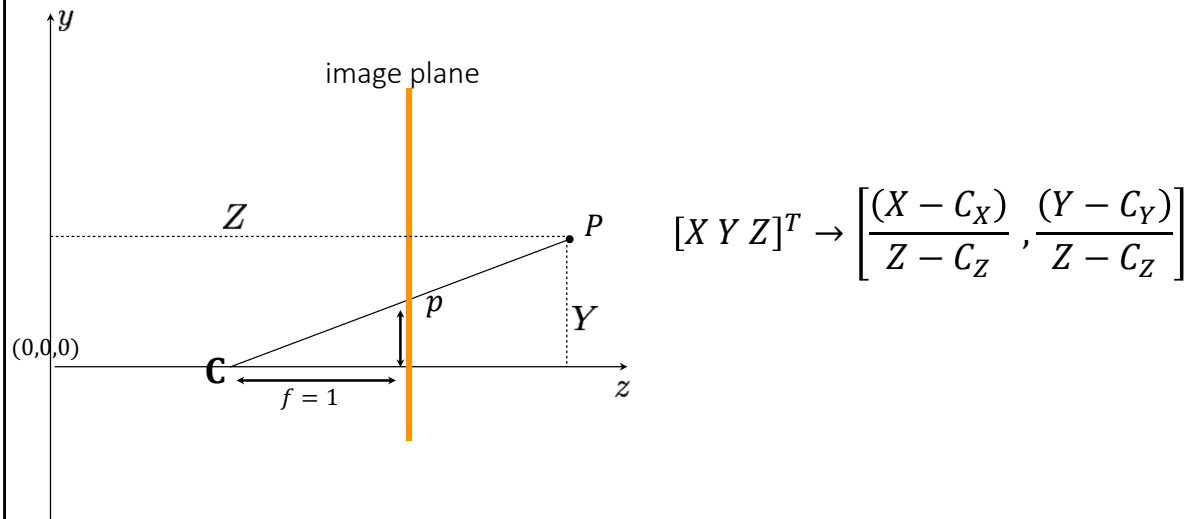
However, we may be using our camera within an application that provides its own world coordinate system. This may be, for example, a GPS or other navigation system used by the robot that the camera is mounted on. To be able to use the projection matrices we have already derived, we need to first convert 3D points from this world coordinate system to the camera coordinate system.

A 2D view of the pinhole camera

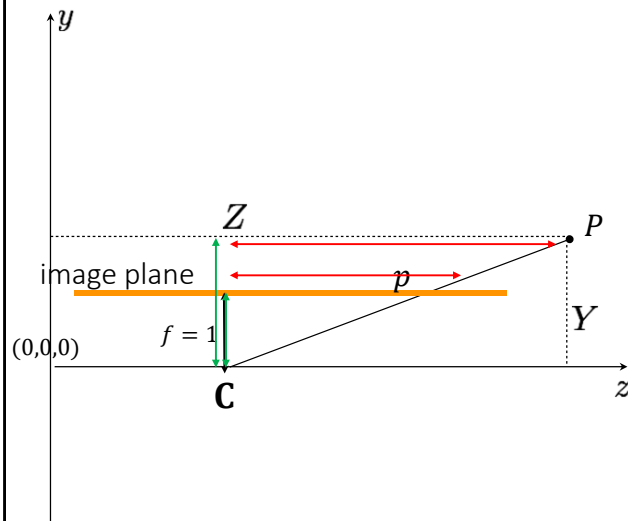


$$[X \ Y \ Z]^T \rightarrow \left[\frac{X}{Z}, \frac{Y}{Z} \right]$$

What if the camera is not located at the origin?



What if the camera is rotated?



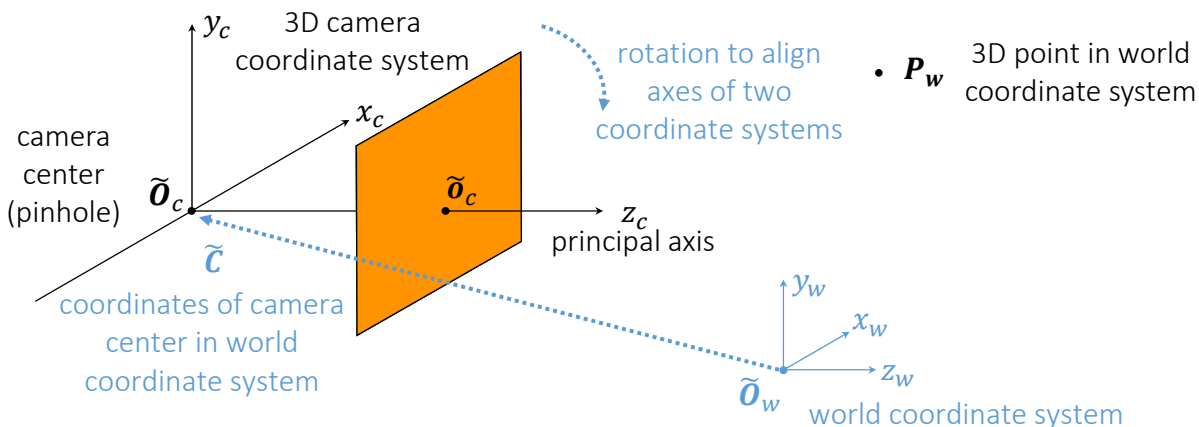
We need to rotate our coordinate system

$$[X \ Y \ Z]^T \rightarrow \left[\frac{(X - C_X)}{Y - C_Y}, \frac{(Z - C_Z)}{Y - C_Y} \right]$$

(For reference, previous formula)

$$[X \ Y \ Z]^T \rightarrow \left[\frac{(X - C_X)}{Z - C_Z}, \frac{(Y - C_Y)}{Z - C_Z} \right]$$

World-to-camera coordinate system transformation



How do we convert between the two 3D coordinate systems?

$$R \cdot (P_w - \tilde{C})$$

rotate translate

In the more general case we want to model how the camera relates to the world around it.

The camera matrices we derived so far assume that the 3D points we project on the image plane are expressed in this camera-based 3D coordinate system.

However, we may be using our camera within an application that provides its own world coordinate system.

We start by expressing this conversion using heterogeneous coordinates. First, we need to translate the point by a vector that shifts the origin of the world coordinate system to match the camera center.

Second, we need to rotate the translated point, by a matrix that aligns the axes of the world and camera coordinate systems.

Modeling the coordinate system transformation

In heterogeneous coordinates, we have:

$$\tilde{\mathbf{P}}_c = \mathbf{R} \cdot (\tilde{\mathbf{P}}_w - \tilde{\mathbf{C}})$$

ب
! 2.3h
3.9h

How do we write this transformation in homogeneous coordinates?

Let's see how we can do this conversion analytically. We start by using heterogeneous coordinates. First, we need to translate the point by a vector that shifts the origin of the world coordinate system to match the camera center.

Second, we need to rotate the translated point, by a matrix that aligns the axes of the world and camera coordinate systems.

The overall transformation uses a translation vector $\mathbf{C}$ and a rotation matrix $\mathbf{R}$, which are determined by how our camera is placed relative to the world coordinate system. We call these the extrinsics of the camera, as they are properties determined by world characteristics external to the camera.

Because we used heterogeneous coordinates, we cannot describe the transformation from the world to the 2D camera coordinate system as simply a

matrix vector product. This can complicate our derivations.

As we saw in the lecture on 2D transformations moving to homogeneous coordinates can save us.

Modeling the coordinate system transformation

In heterogeneous coordinates, we have:

$$\tilde{\mathbf{P}}_c = \mathbf{R} \cdot (\tilde{\mathbf{P}}_w - \tilde{\mathbf{C}})$$

In homogeneous coordinates, we have:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & -\mathbf{R}\tilde{\mathbf{C}} \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad \text{or} \quad \mathbf{P}_c = \begin{bmatrix} \mathbf{R} & -\mathbf{R}\tilde{\mathbf{C}} \\ \mathbf{0} & 1 \end{bmatrix} \mathbf{P}_w$$

Notations:

_w subscript- world coordinate system

_c subscript- camera coordinate system

we can overcome this problem using homogeneous coordinates. Then, we can write the mapping from the world to the camera coordinate system as a matrix vector product. The 4x4 transformation matrix is expressed in terms of the rotation matrix and translation vector we used with heterogeneous coordinates.

Incorporating the transform in the camera matrix

The previous camera matrix is for homogeneous 3D coordinates in camera coordinate system:

$$\mathbf{p} = \mathbf{M}\mathbf{P}_c = \mathbf{K}[\mathbf{I}|\mathbf{0}]\mathbf{P}_c \quad \mathbf{M} = \mathbf{K}[\mathbf{I}|\mathbf{0}] = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

We also just derived:

$$\mathbf{P}_c = \begin{bmatrix} \mathbf{R} & -\mathbf{R}\tilde{\mathbf{C}} \\ \mathbf{0} & 1 \end{bmatrix} \mathbf{P}_w$$

Now let's see how all this fits into our camera matrix model. We have derived previously a matrix that maps 3D homogeneous points expressed in the camera coordinate system to 2D points on the image plane. We have also just derived a matrix that maps 3D homogeneous points from the world to the camera coordinate system.

Putting it all together

We can write everything into a single projection:

$$\mathbf{p} = \mathbf{M}\mathbf{P}_w$$

The camera matrix now looks like:

$$\mathbf{M} = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{R} & -\mathbf{R}\tilde{\mathbf{C}} \\ \mathbf{0} & 1 \end{bmatrix}$$

intrinsic parameters (3 x 3):
correspond to camera
internals (2D image-to-image
transformation)

perspective projection (3 x 4):
maps 3D to 2D points
(camera-to-image
transformation)

extrinsic parameters (4 x 4):
correspond to camera
externals (3D world-to-camera
transformation)

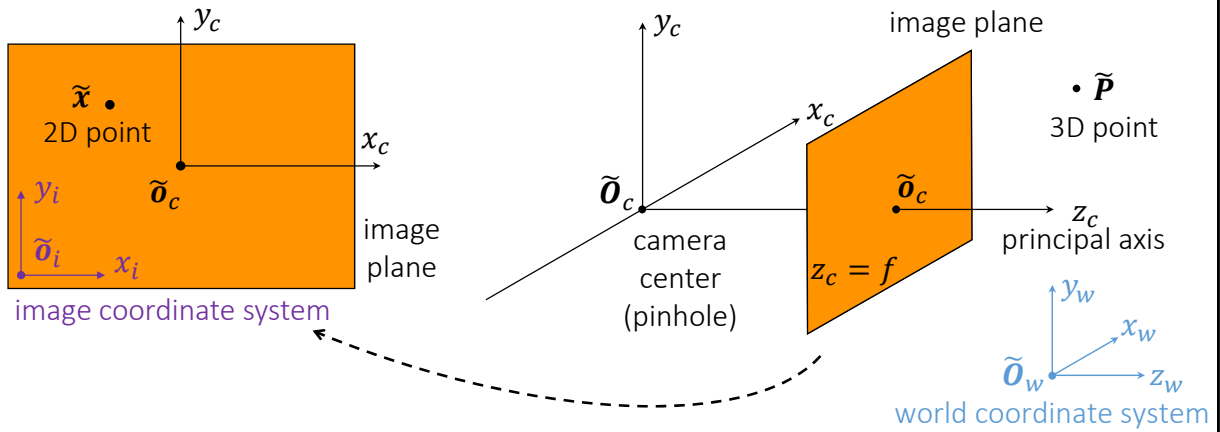
We can put all of this together into a new camera matrix that directly maps 3D points in the world coordinate system to 2D points on the image plane. Our updated camera matrix is now the composition of three matrices.

The middle matrix is nothing else but the 3x4 perspective projection matrix that we derived earlier, when we started describing the pinhole camera.

The left matrix, called the camera intrinsics, is a 3x3 matrix corresponding to a 2D image-to-image transformation. This matrix depends on only the internal properties of the camera.

Lastly, the right matrix, called the camera extrinsics, is a 4x4 matrix corresponding to a 3D transformation from the world to the camera coordinate system. This matrix depends on only the external properties of the camera.

Mapping world camera and image coordinates



2D camera coordinate system

3D camera coordinate system

- A camera introduces two related coordinate systems, in 3D (world), and in 2D (image plane).
- These coordinate systems may be different from the coordinate systems of our application.

In this visual diagram, we can view the camera matrix as a sequence of three transformations. First, we convert from the world to the camera 3D coordinate system. Then, we project from the camera 3D coordinate system to the image plane. Lastly, we convert from the image plane to a different image coordinate system.

Simplifying notation

We can write everything into a single projection:

$$p = MP_w$$

The camera matrix now looks like:

$$M = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} [R] \quad -R\tilde{C}$$

It is common to combine the perspective projection and extrinsics in one matrix.

intrinsic parameters (3 x 3):
correspond to camera
internals (2D image-to-image
transformation)

extrinsic parameters (3 x 4):
correspond to camera externals (3D
world-to-camera transformation)
and perspective projection

I wanted to highlight two notational conventions that are commonplace in computer vision literature and software systems. First, very often we combine the 3x4 perspective projection matrix and the 4x4 3D transformation matrix into a single 3x4 matrix, which we also called the camera extrinsics.

Simplifying notation

More compactly, we can write the pinhole camera matrix as:

$$\mathbf{M} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$$

where

$$\mathbf{K} = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{R} = \begin{bmatrix} r_1 & r_2 & r_3 \\ r_4 & r_5 & r_6 \\ r_7 & r_8 & r_9 \end{bmatrix} \quad \mathbf{t} = -\mathbf{R}\tilde{\mathbf{C}} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix}$$

2D Euclidean transform 3D rotation 3D translation

intrinsic parameters extrinsic parameters

Second, we commonly use the symbols $\mathbf{K}$, $\mathbf{R}$, and $\mathbf{t}$, for the intrinsic matrix, the extrinsic rotation matrix, and the extrinsic rotated translation vector. This notation makes it possible to write the camera matrix very compactly.

General pinhole camera matrix

We can decompose the camera matrix like this:

$$\mathbf{M} = \mathbf{K}\mathbf{R}[\mathbf{I} | -\mathbf{C}]$$

(translate first then rotate)

Another way to write the mapping:

$$\mathbf{M} = \mathbf{K} [\mathbf{R} | \mathbf{t}]$$

where $\mathbf{t} = -\mathbf{R}\mathbf{C}$

(rotate first then translate)

General pinhole camera matrix

$$M = KR[I| - C] \quad (\text{translate first then rotate})$$

$$M = K [R|t] \quad (\text{rotate first then translate})$$

$$\text{where } \mathbf{t} = -\mathbf{RC}$$

Q: What is the projection of the camera center C ?

$$A: MC = 0$$

We can show that the camera center is projected to the vector $[0,0,0]$. Or C is the null space of M .

The easy way to see this is just from the equation if we multiply $[I|-C][C;1]$ we get $C-C=0$.

The geometric interpretation is as follows: to project a point into the image we connect it to the center of the camera and check where this line intersects the image plane.

However, to define a line we need 2 different points. If the point we want to project is the camera center, the line connecting it to the camera center is not defined. Thus the projection of the camera center is ill defined. Remember that in homogenous coordinates $[0;0;0]$ is the one undefined vector.

The fact that $MC=0$ is very important and we will reuse it later in this lecture as well as in the following ones.

Quiz

What is the size and meaning of each term in the camera matrix?

$$M = KR[I] - C$$

The diagram shows the equation $M = KR[I] - C$ with four arrows pointing from labels below to terms in the equation:

- An arrow points from "3x3 intrinsics" to K .
- An arrow points from "3x3 3D rotation" to R .
- An arrow points from "3x3 identity" to $[I]$.
- An arrow points from "3x1 3D translation" to C .

Quiz

The camera matrix relates what two quantities?

$$p = MP$$

homogeneous 3D points to homogeneous 2D image points

The camera matrix can be decomposed into?

$$M = K [R|t]$$

intrinsic and extrinsic parameters

Quiz: what is the difference between the following translation operators?

Translation of 2D projection.

$$M = \begin{bmatrix} 1 & 0 & m_x & 0 \\ 0 & 1 & m_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & m_x \\ 0 & 1 & m_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Translation amount does not depend on depth z.

$$[X \ Y \ Z]^T \rightarrow \left[\frac{X + Zm_x}{Z}, \frac{Y + Zm_y}{Z} \right] = \left[\frac{X}{Z} + m_x, \frac{Y}{Z} + m_y \right]$$

$$M = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Translation of 3D position.

$[X \ Y \ Z]^T \rightarrow$

$$\left[\frac{X + t_x}{Z}, \frac{Y + t_y}{Z} \right] = \left[\frac{X}{Z} + \frac{t_x}{Z}, \frac{Y}{Z} + \frac{t_y}{Z} \right]$$

Translation amount depends on depth z.

More general camera matrices

The following is the standard camera matrix we saw.

$$M = \begin{bmatrix} f & 0 & m_x \\ 0 & f & m_y \\ 0 & 0 & 1 \end{bmatrix} [R | -RC]$$

How many degrees of freedom?

9 DOF

9 parameters=
3 parameters for translation
3 parameters for rotation
3 intrinsic parameters

A rotation matrix in 3D can be defined by 3 parameters: rotation axis which is a vector on the unit sphere (2 parameters), plus the angle by which we rotate around it (1 parameter)

We have so far described a camera matrix for a pinhole camera of arbitrary focal length and image coordinate system, and placed at an arbitrary location and pose within some external world coordinate system.

We can produce derive matrices for even more general camera models, by modifying the intrinsics matrix to account for additional internal characteristics of the camera.

More general camera matrices

CCD camera: pixels may not be square.

$$M = \begin{bmatrix} f_x & 0 & m_x \\ 0 & f_y & m_y \\ 0 & 0 & 1 \end{bmatrix} [R | -RC]$$

How many degrees of freedom?

10 DOF

For example, pixels may not be square. We can account for this by modifying the diagonal of the intrinsics matrix to have different scale factors for x and y coordinates. Both of these scale factors will depend on the focal length of the camera.

More general camera matrices

Finite projective camera: sensor be skewed.

$$M = \begin{bmatrix} f_x & s & m_x \\ 0 & f_y & m_y \\ 0 & 0 & 1 \end{bmatrix} [R | -RC]$$

How many degrees of freedom?

11 DOF

Because of manufacturing errors, the sensor of a camera may end up being skewed. We can again account for this, by modifying the intrinsics matrix to include, in addition to scaling a translation, a skew factor s . This updated camera matrix is the most general matrix we can derive for cameras that implement perspective projection. We refer to the corresponding camera model as the finite projective camera.

More general camera matrices

$$M = \begin{bmatrix} f_x & s & m_x \\ 0 & f_y & m_y \\ 0 & 0 & 1 \end{bmatrix} [R | -RC]$$

11 DOF

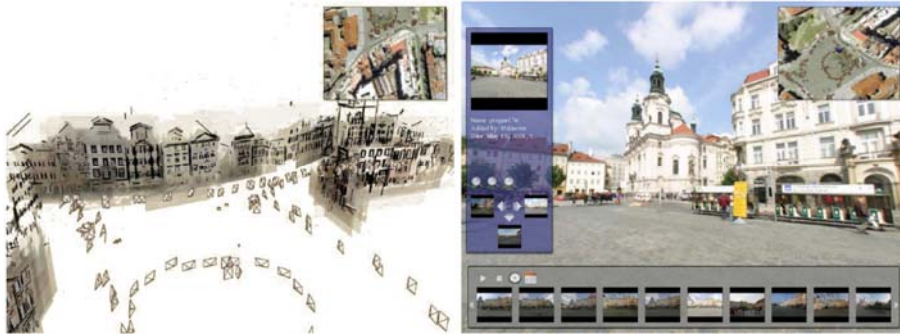
Effectively: M is a 3x4=12 elements matrix defined up to scale (homogeneous coordinates)

(Will allow linearizing the estimation problem)

Geometric camera calibration

	Structure (scene geometry)	Motion (camera geometry) <i>M Z-hat</i>	Measurements
Camera Calibration (a.k.a. Pose Estimation)	known <i>{P_i, q_i}</i> <i>new</i>	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D correspondences

Pose Estimation



Given a single image,
estimate the exact position of the photographer

We have now seen how to use a 3×4 camera matrix to describe how a camera maps 3D points in the world, to 2D points on an image. We will now answer the following question: how can I determine the camera matrix of some arbitrary camera I am using? This task is called geometric camera calibration.

Geometric camera calibration

Given a set of matched points

$$\{P_i, p_i\}$$

point in 3D space point in the image

and camera model

$$p = \underset{\substack{\text{projection} \\ \text{model}}}{f}(P; \underset{\text{parameters}}{m}) = \underset{\substack{\text{Camera} \\ \text{matrix}}}{M} P$$

Find the (pose) estimate of

$$M$$

We'll use a **perspective** camera model for pose estimation

To be able to calibrate the camera, we will need to have available a set of matched pairs between 2D points on an image captured by the camera, and the corresponding 3D points on the scene that the image was taken from.

Same setup as homography estimation
(slightly different derivation here)

Mapping between 3D point and image points

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

What are the unknowns?

To see how to perform calibration, let's first write out the mapping between one matched pair of 3D and 2D points. In homogeneous coordinates, this mapping is done through the unknown camera matrix, that has 12 entries.

Mapping between 3D point and image points

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} - & m_1^T & - \\ - & m_2^T & - \\ - & m_3^T & - \end{bmatrix} P$$

Heterogeneous coordinates

$$x = \frac{m_1^T P}{m_3^T P} \quad y = \frac{m_2^T P}{m_3^T P}$$

(non-linear relation between coordinates)

How can we make these relations linear?

To make notation a little simpler, we will rewrite the camera matrix in terms of three 4x1 vectors, each equal to the transpose of one of the camera matrix rows.

What we have available as our input data are the heterogeneous coordinates of the 2D point. Expressing them in terms of the 3D point and the row-vectors of the camera matrix gives us two non-linear relationships that are hard to manipulate.

How can we make these relations linear?

$$x = \frac{m_1^T P}{m_3^T P} \quad y = \frac{m_2^T P}{m_3^T P}$$

Make them linear with algebraic manipulation...

$$m_1^T P - x m_3^T P = 0$$

$$m_2^T P - y m_3^T P = 0$$

Now we can setup a system of linear equations with multiple point correspondences

We can convert these into two linear relationships, simply rearranging some of the terms.

$$m_1^T P - x m_3^T P = 0$$

$$m_2^T P - y m_3^T P = 0$$

How do we proceed?

$$m_1^T P - x m_3^T P = 0$$

$$m_2^T P - y m_3^T P = 0$$

In matrix form ...

$$\begin{bmatrix} P^T & 0 & -x P^T \\ 0 & P^T & -y P^T \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix} = 0$$

$\begin{matrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \end{matrix}$
 $\begin{matrix} 2 \times 12 \\ 12 \times 1 \end{matrix}$

How many rows x columns?

We can then rewrite these relationships in matrix form: For this, we use a 2x12 matrix that depends on the 3D and 2D coordinates of the matched pair of points; and a 12x1 vector of the entries of the unknown camera matrix.

$$m_1^T P - x m_3^T P = 0$$

$$m_2^T P - y m_3^T P = 0$$

In matrix form ...

$$\begin{bmatrix} P^T & 0 & -x P^T \\ 0 & P^T & -y P^T \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix} = 0$$

How do we proceed?

$$m_1^T P - x m_3^T P = 0$$

$$m_2^T P - y m_3^T P = 0$$

In matrix form ...

$$\begin{bmatrix} P^T & 0 & -xP^T \\ 0 & P^T & -yP^T \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix} = 0$$

For N points ...

$$\begin{bmatrix} P_1^T & 0 & -xP_1^T \\ 0 & P_1^T & -yP_1^T \\ \vdots & \vdots & \vdots \\ P_N^T & 0 & -xP_N^T \\ 0 & P_N^T & -yP_N^T \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix} = 0$$

*How do we solve
this system?*

We can stack together the 2x12 matrices corresponding to all matched pairs that we have available as data, to produce a homogeneous linear system with the camera matrix entries as unknowns.

Solve for camera matrix by

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x}\|^2 \text{ subject to } \|\mathbf{x}\|^2 = 1$$

$$A = \begin{bmatrix} P_1^T & 0 & -xP_1^T \\ 0 & P_1^T & -yP_1^T \\ & \vdots & \\ P_N^T & 0 & -xP_N^T \\ 0 & P_N^T & -yP_N^T \end{bmatrix} \quad x = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix}$$

SVD!

Lastly, and as we have seen in the lecture on 2D Transformations and Homographies, we can use the singular value decomposition to solve this homogeneous linear system.

Solve for camera matrix by

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x}\|^2 \text{ subject to } \|\mathbf{x}\|^2 = 1$$

$$A = \begin{bmatrix} P_1^T & 0 & -xP_1^T \\ 0 & P_1^T & -yP_1^T \\ & \vdots & \\ P_N^T & 0 & -xP_N^T \\ 0 & P_N^T & -yP_N^T \end{bmatrix} \quad x = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix}$$

Solution $\mathbf{x}$ is the column of $\mathbf{V}$
corresponding to smallest singular
value of

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

Solve for camera matrix by

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x}\|^2 \text{ subject to } \|\mathbf{x}\|^2 = 1$$

$$A = \begin{bmatrix} P_1^T & 0 & -xP_1^T \\ 0 & P_1^T & -yP_1^T \\ & \vdots & \\ P_N^T & 0 & -xP_N^T \\ 0 & P_N^T & -yP_N^T \end{bmatrix} \quad x = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix}$$

solution $\mathbf{x}$ is the Eigenvector
corresponding to smallest
Eigenvalue of

$$\mathbf{A}^T \mathbf{A}$$

Reshape the 12 elements vector into a 3x4 matrix

$$M = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix}$$

Are we done?

Solving the linear system gives us the unknown 3x4 camera matrix P. However, we are not yet done with geometric calibration.

Almost there ... $M = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix}$

How do you get the intrinsic and extrinsic parameters from the projection matrix?

We need to decompose this matrix into the intrinsic and extrinsic components.

Decomposition of the Camera Matrix

$$M = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix}$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

To do this, we will slightly rearrange the various terms in the decomposition expression, and rewrite the camera matrix in terms of a 3x3 matrix N , and the 3x1 camera center c .

Decomposition of the Camera Matrix

$$M = \left[\begin{array}{ccc|c} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{array} \right]$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

What is the projection of the camera center?

Find intrinsic **K** and rotation **R**

Our goal now is to compute the camera center c , which is nothing else but the location of the camera pinhole, and the matrix N that depends on the intrinsic matrix K and extrinsic rotation matrix R .

To compute the camera center c , we can take advantage of a property of the camera center, namely, that the camera matrix projects it to the zero vector. Therefore, we can compute the singular value decomposition of the camera matrix M , and set c to be equal to the singular vector corresponding to the smaller singular value.

Decomposition of the Camera Matrix

$$M = \left[\begin{array}{ccc|c} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{array} \right]$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

$$Mc = 0$$

How do we compute the camera center from this?

Find intrinsic **K** and rotation **R**

Decomposition of the Camera Matrix

$$M = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix}$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

$$Mc = 0$$

SVD of M!

c is the singular vector corresponding to smallest singular value

Find intrinsic **K** and rotation **R**

Decomposition of the Camera Matrix

$$M = \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix}$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

$$Mc = 0$$

SVD of M!

c is the singular vector corresponding to smallest singular value

Find intrinsic **K** and rotation **R**

$$N = KR$$

Any useful properties of K and R we can use?

To compute the matrices K and R, we will take advantage of the two of their properties: The intrinsic matrix K is, as we saw in a previous submodule, upper triangular. And the extrinsic rotation matrix R is orthogonal, as are all rotation matrices.

Therefore, we can estimate these two matrices by computing the QR decomposition of N. This is a standard linear algebra algorithm that uniquely factorizes any matrix into the product of an upper triangular matrix and an orthogonal matrix---just like the matrices K and R we want to estimate. With this last step, we have completely calibrated all of the geometric characteristics of our camera.

Decomposition of the Camera Matrix

$$M = \left[\begin{array}{ccc|c} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{array} \right]$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

$$Mc = 0$$

SVD of M!

c is the Eigenvector corresponding to smallest Eigenvalue

Find intrinsic **K** and rotation **R**

$$N = KR$$

right upper triangle orthogonal

How do we find K and R?

Decomposition of the Camera Matrix

$$M = \left[\begin{array}{ccc|c} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{array} \right]$$

$$\begin{aligned} M &= K[R|t] \\ &= K[R|-Rc] \\ &= [N| -Nc] \end{aligned}$$

Find the camera center **C**

$$Mc = 0$$

SVD of M!

*c is the Eigenvector corresponding to
smallest Eigenvalue*

Find intrinsic **K** and rotation **R**

$$N = KR$$

QR decomposition

Geometric camera calibration

Given a set of matched points

$$\{P_i, p_i\}$$

point in 3D
space

point in the
image

*Where do we get these
matched points from?*

and camera model

$$p = f(P; m) = M P$$

projection
model

parameters

Camera
matrix

Find the (pose) estimate of

$$M$$

We'll use a **perspective** camera
model for pose estimation

Going back to the problem setup for camera calibration, we assumed that we have available as input a set of matched 3D and 2D points. How do we obtain these matched points in practice?

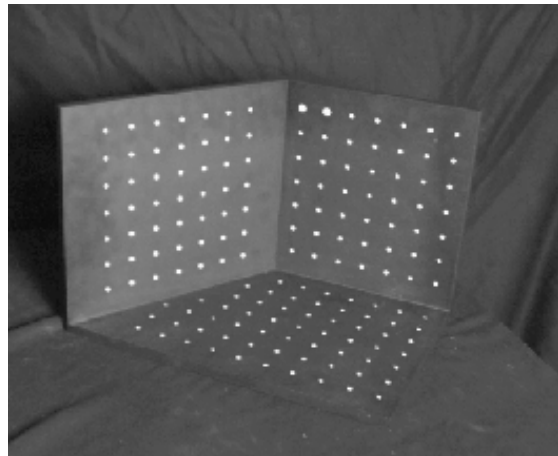
Calibration using a reference object

Place a known object in the scene:

- identify correspondences between image and scene
- compute mapping from scene to image

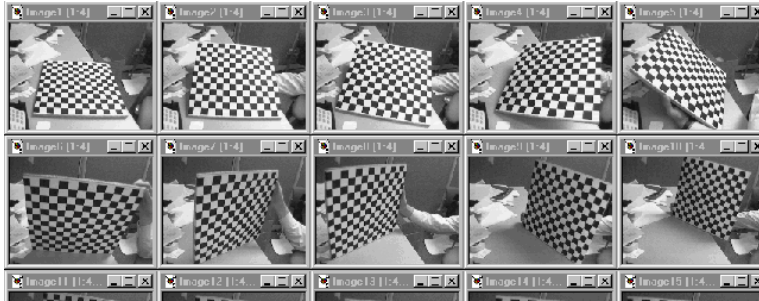
Issues:

- must know geometry very accurately
- must know 3D->2D correspondence



In the past, to get sets of matched points, it was necessary to have available some accurately manufactured 3D calibration object, and take a photograph of it with the camera we wanted to calibrate.

Alternative: Multi-plane calibration



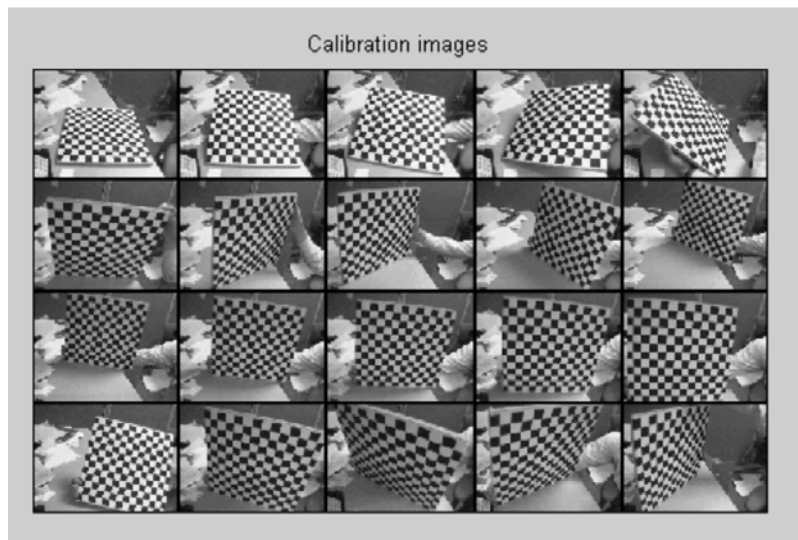
Advantages:

- Only requires a plane
- Don't have to know positions/orientations
- also corrects lens radial distortion
- Great code available online!
 - Matlab version: http://www.vision.caltech.edu/bouguetj/calib_doc/index.html
 - Also available on OpenCV.

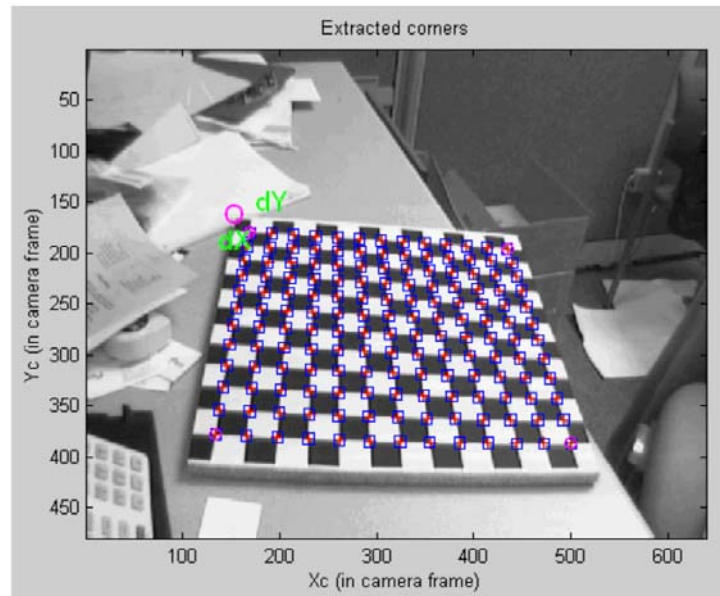
Disadvantage: Need to solve non-linear optimization problem.

Nowadays, the calibration procedure is a lot simpler: All we have to do is print a checkerboard pattern, attach it to a planar surface, and then take multiple photos of this checkerboard under very many different poses and locations. There are online a lot of robust and easy-to-use toolboxes implementing this calibration procedure. I encourage you to try it out on your own camera, for example the camera on your cell phone!

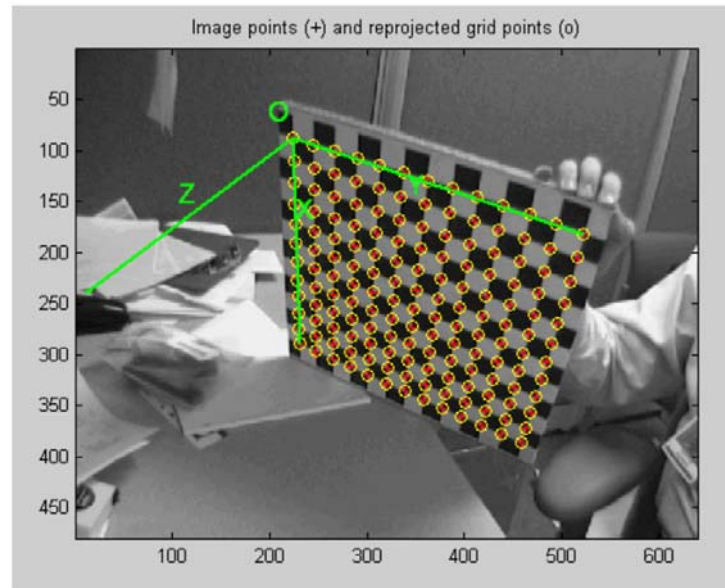
Step-by-step demonstration



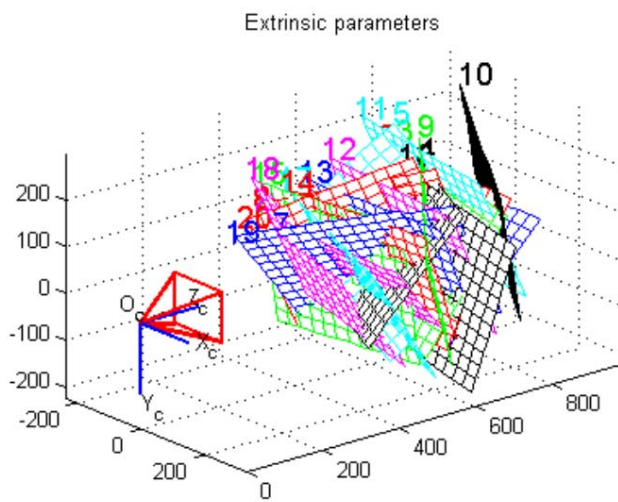
Step-by-step demonstration



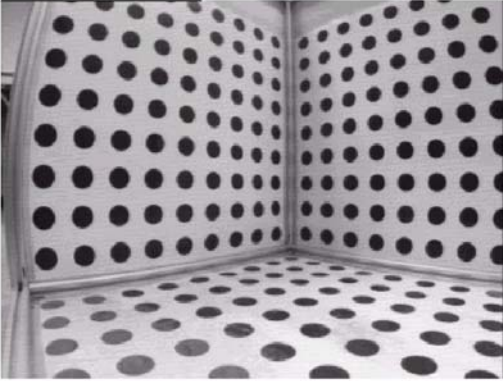
Step-by-step demonstration



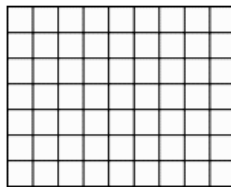
Step-by-step demonstration



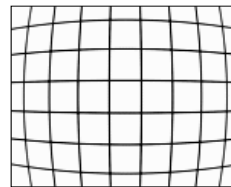
Radial distortion



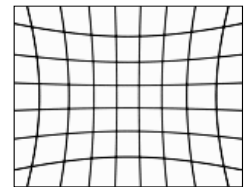
What causes this distortion?



no distortion



barrel distortion



pincushion distortion

In addition to calibrating the camera matrix, these toolboxes can also correct a type of image artifact called radial distortion. This is unrelated to perspective distortion, and is caused by imperfections in lens manufacturing. We will not go into details of how this correction is done, but it is useful to know this and keep it in mind nonetheless.

Correcting radial distortion

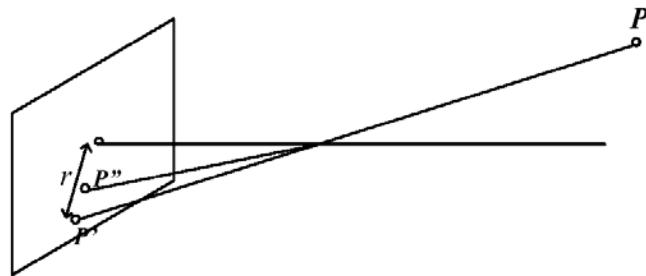


before



after

Radial distortion model



Ideal:

$$x' = f \frac{x}{z}$$

$$y' = f \frac{y}{z}$$

Distorted:

$$x'' = \frac{1}{\lambda} x'$$

$$y'' = \frac{1}{\lambda} y'$$

$$\lambda = 1 + k_1 r^2 + k_2 r^4 + \dots$$

Linear camera calibration

Advantages:

- Very simple to formulate.
- Analytical solution.(use SVD)

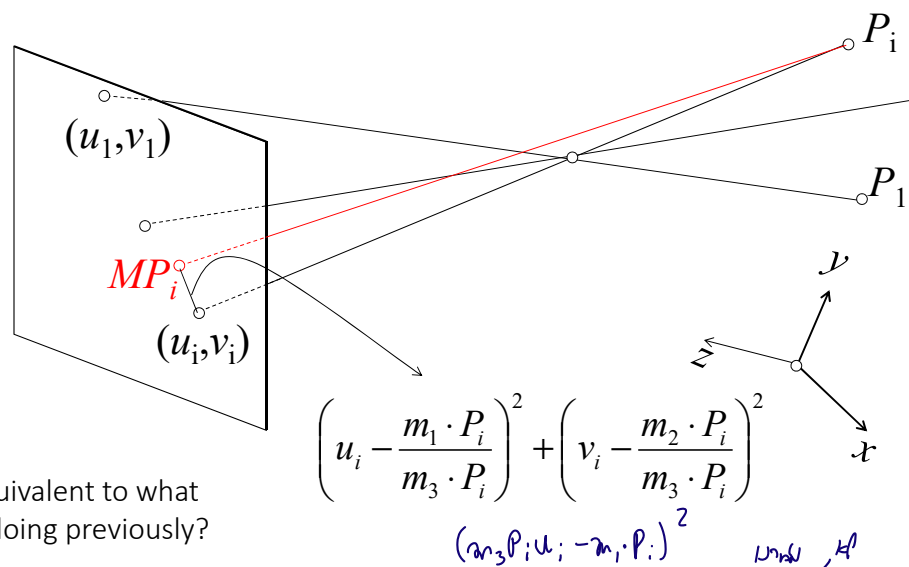
Disadvantages:

- Doesn't model radial distortion.
- Hard to impose constraints (e.g., known f).
- Doesn't minimize the correct error function.

For these reasons, *nonlinear methods* are preferred

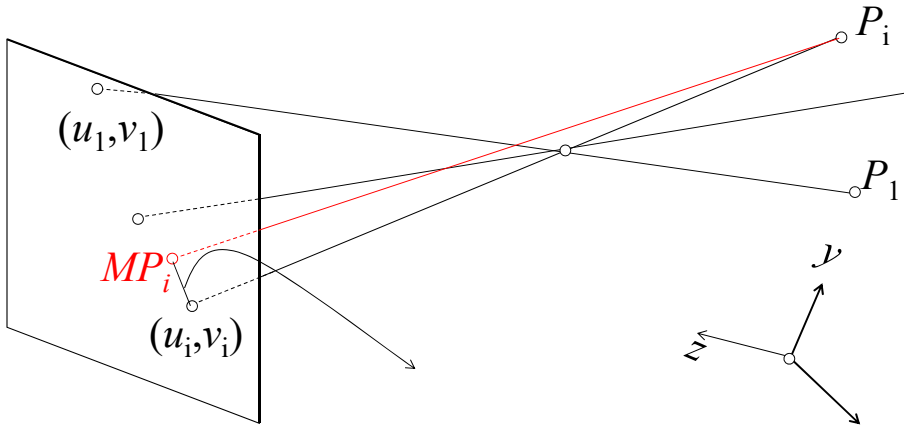
- Define error function E between projected 3D points and image positions
 - E is nonlinear function of intrinsics, extrinsics, radial distortion
- Minimize E using nonlinear optimization techniques

Minimizing reprojection error



Is this equivalent to what we were doing previously?

Minimizing reprojection error with radial distortion



Add distortions to reprojection error:

$$\left(u_i - \frac{1}{\lambda} \frac{m_1 \cdot P_i}{m_3 \cdot P_i}\right)^2 + \left(v_i - \frac{1}{\lambda} \frac{m_2 \cdot P_i}{m_3 \cdot P_i}\right)^2$$

What does it mean to “calibrate a camera”?

Many different ways to calibrate a camera:

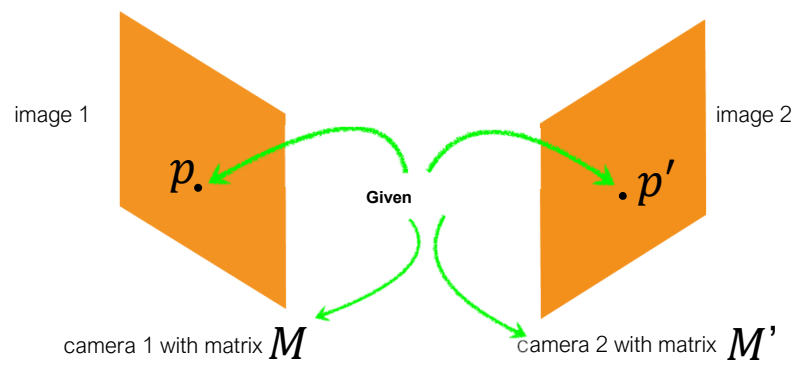
- Radiometric calibration.
- Color calibration.
- Geometric calibration.
- Noise calibration.
- Lens (or aberration) calibration.



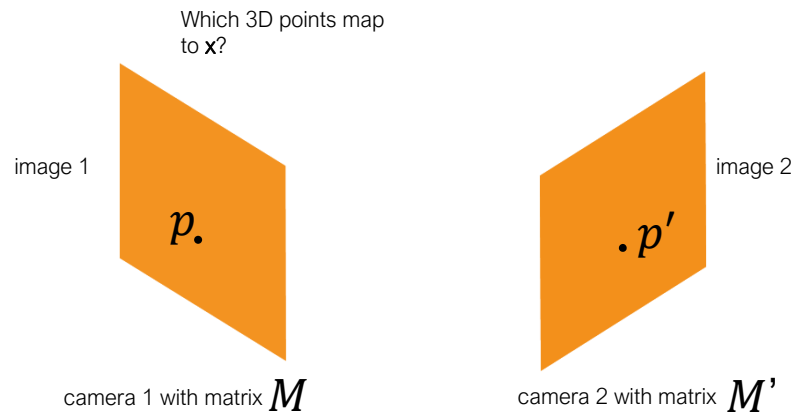
	Structure (scene geometry)	Motion (camera geometry)	Measurements
Camera Calibration (a.k.a. Pose Estimation)	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D coorespondences

Triangulation

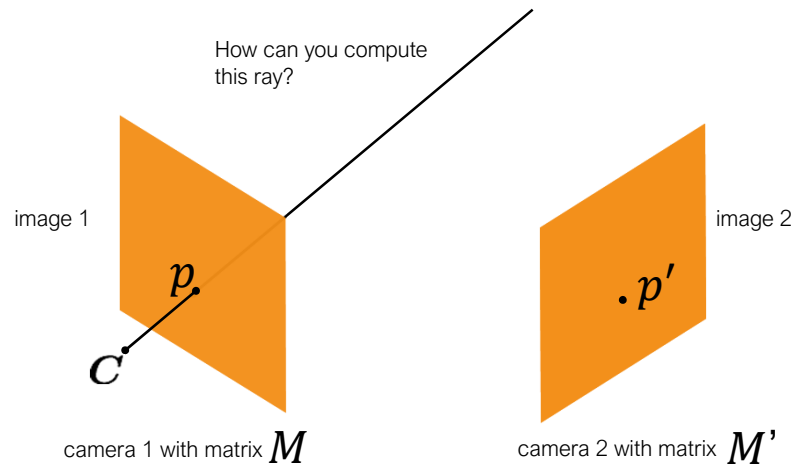
Triangulation



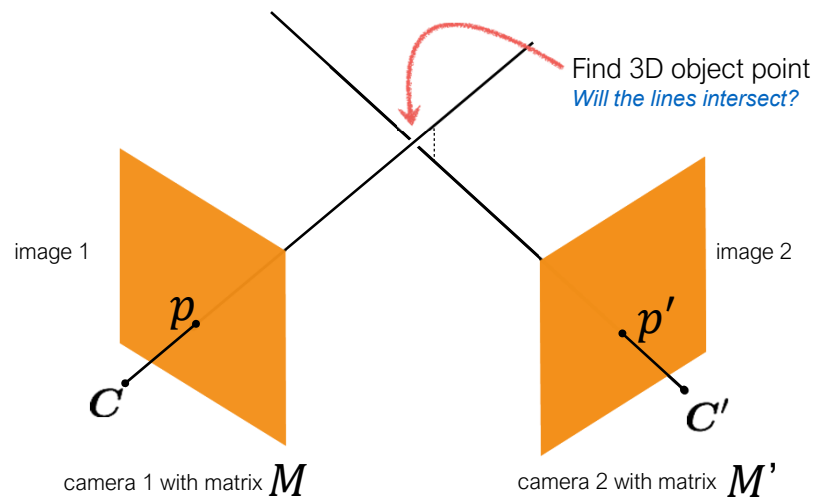
Triangulation



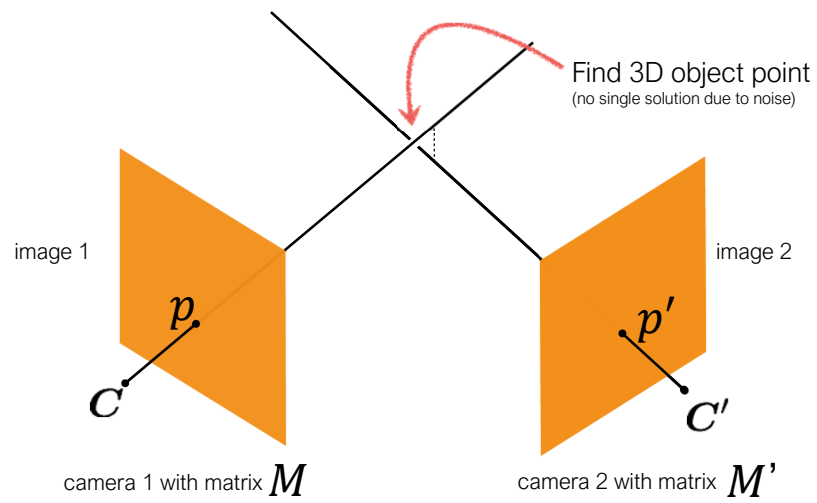
Triangulation



Triangulation



Triangulation



$$p = MP$$

(homogeneous
coordinate)

Also, this is a similarity relation because it involves homogeneous coordinates

$$p = \alpha MP$$

(homogeneous
coordinate)

Same ray direction but differs by a scale factor

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

How do we solve for unknowns in a similarity relation?

$$p = MP$$

(homogeneous
coordinate)

Also, this is a similarity relation because it involves homogeneous coordinates

$$p = \alpha MP$$

(homogeneous
coordinate)

Same ray direction but differs by a scale factor

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

How do we solve for unknowns in a similarity relation?

Remove scale factor, convert to linear system and solve with



$$p = MP$$

(homogeneous
coordinate)

Also, this is a similarity relation because it involves homogeneous coordinates

$$p = \alpha MP$$

(homogeneous
coordinate)

Same ray direction but differs by a scale factor

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

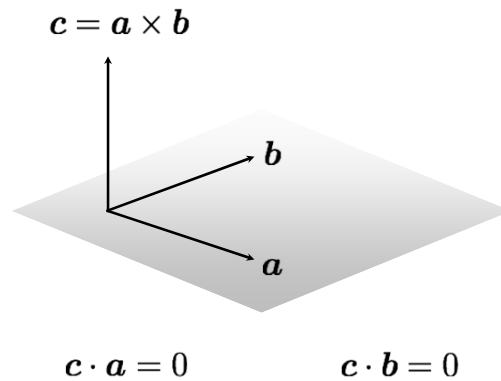
How do we solve for unknowns in a similarity relation?

Remove scale factor, convert to linear system and solve with SVD!

Recall: Cross Product

Vector (cross) product

takes two vectors and returns a vector perpendicular to both



$$a \times b = \begin{bmatrix} a_2b_3 - a_3b_2 \\ a_3b_1 - a_1b_3 \\ a_1b_2 - a_2b_1 \end{bmatrix}$$

cross product of two vectors in the same direction is zero

$$a \times a = 0$$

remember this!!!

$$p = \alpha MP$$

Same direction but differs by a scale factor

$$p \times MP = 0$$

Cross product of two vectors of same direction is zero
(this equality removes the scale factor)

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} - & m_1^T & - \\ - & m_2^T & - \\ - & m_3^T & - \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1 & m_2 & m_3 & m_4 \\ m_5 & m_6 & m_7 & m_8 \\ m_9 & m_{10} & m_{11} & m_{12} \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} - & m_1^T & - \\ - & m_2^T & - \\ - & m_3^T & - \end{bmatrix} P$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \alpha \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \times \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix} = \begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \\ xm_2^T P - ym_1^T P \end{bmatrix} = 0$$

Using the fact that the cross product should be zero

$$p \times MP = 0$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \times \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix} = \begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \\ xm_2^T P - ym_1^T P \end{bmatrix} = 0$$

Third line is a linear combination of the first and second lines.
(x times the first line plus y times the second line)

One 2D to 3D point correspondence give you  equations

Using the fact that the cross product should be zero

$$p \times MP = 0$$

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \times \begin{bmatrix} m_1^T P \\ m_2^T P \\ m_3^T P \end{bmatrix} = \begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \\ xm_2^T P - ym_1^T P \end{bmatrix} = 0$$

Third line is a linear combination of the first and second lines.
(x times the first line plus y times the second line)

One 2D to 3D point correspondence give you 2 equations

$$\begin{bmatrix} ym_3^T P - m_2^T P \\ m_1^T P - xm_3^T P \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} ym_3^T - m_2^T \\ m_1^T - xm_3^T \end{bmatrix} P = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$A_i P = 0$$

Now we can make a system of linear equations
(two lines for each 2D point correspondence)

Concatenate the 2D points from both images

$$\begin{bmatrix} ym_3^T - m_2^T \\ m_1^T - xm_3^T \\ y'm_3'^T - m_2'^T \\ m_1'^T - x'm_3'^T \end{bmatrix} P = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

sanity check! dimensions?

$$AP = 0$$

How do we solve homogeneous linear system?

Concatenate the 2D points from both images

$$\begin{bmatrix} ym_3^T - m_2^T \\ m_1^T - xm_3^T \\ y'm_3'^T - m_2'^T \\ m_1'^T - x'm_3'^T \end{bmatrix} P = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$AP = 0$$

How do we solve homogeneous linear system?

S V D !

	Structure (scene geometry)	Motion (camera geometry)	Measurements
Camera Calibration (a.k.a. Pose Estimation)	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D correspondences

References

Basic reading:

- Szeliski textbook, Section 2.1.5, 6.2.

Additional reading:

- Hartley and Zisserman, "Multiple View Geometry in Computer Vision," Cambridge University Press 2004. chapter 6 of this book has a very thorough treatment of camera models.
- Torralba and Freeman, "Accidental Pinhole and Pinspeck Cameras," CVPR 2012. the eponymous paper discussed in the slides.